JavaScript Fundamentals — Crash Course Notes

1. JavaScript Kya Hai?

- JavaScript ek **high-level, interpreted language** hai.
  - High-level: memory management jaisi cheezein khud handle nahi karni parti (C/C++ ki tarah low-level nahi hai).
  - Interpreted: compiler ke through run nahi hota, direct execute hota hai (Java ek compiled language hai, alag cheez hai).
- JavaScript **ECMAScript specification** ko follow karti hai. Doosre implementations bhi hain (JScript, ActionScript) lekin JS sab se popular hai.
- JavaScript **multi-paradigm** hai — object-oriented ya functional, jaisa marzi likh sakte ho.
- Ye **browser (client)** ki language hai — HTML/CSS ke sath interactivity (form validation, alerts) ke liye use hoti hai.
- **Node.js** ke zariye server-side bhi chal sakti hai (database interaction waghera).
- JS frameworks: React, Angular, Vue.js. Mobile: React Native, NativeScript. Desktop: Electron.

2. Setup & Console

- JS file ko HTML mein link karne ka tareeqa: `<script src="main.js"></script>` — ye **body ke end mein**, closing `</body>` se pehle likhna chahiye (taake HTML/CSS pehle load ho jaye).
- `alert('text')` ek popup deta hai lekin debugging ke liye achha nahi (script block ho jata hai).
- **Console** debugging ka best tareeqa hai:
  - `console.log()` — normal output
  - `console.error()` — red error message
  - `console.warn()` — yellow warning message

- Browser dev tools kholna: Chrome mein `Cmd+Option+I` (Mac) ya `F12` (Windows).

3. Variables — var, let, const

- **var**: purana tareeqa, globally scoped hota hai — conflicts create kar sakta hai (naming clashes block ke bahar/andar). Ab use nahi karte.
- **let**: ES6 (ES2015) mein aaya — value **reassign** ki ja sakti hai.
- **const**: value reassign **nahi** ki ja sakti (constant). Declare karte waqt value dena zaroori hai (initializer chahiye), warna error aayega.
- General rule: **const use karo jab tak reassign nahi karna**, warna `let` use karo (jaise game score jo change hota rahay).
- Arrays/objects ke sath const use kar sakte ho kyunke unke andar values change ho sakti hain, sirf poori variable ko reassign nahi kar sakte.

4. Data Types (Primitive)

Primitive data types wo hain jinki value directly memory mein assign hoti hai:

- **String** — text, single ya double quotes mein (single quotes prefer kiye jate hain)
- **Number** — integers aur decimals dono, JS mein alag se "float" type nahi hota
- **Boolean** — `true` / `false`
- **null** — variable hai lekin andar kuch nahi (intentionally empty)
- **undefined** — variable declare hua hai lekin value assign nahi hui
- **Symbol** — ES6 mein aaya, rarely use hota, is course mein cover nahi kiya

Semicolons JS mein mandatory nahi hain, lekin most developers use karte hain.

`typeof` operator

- `typeof` se data type check karte hain.
- **Gotcha**: `typeof null` → `"object"` deta hai (ye ek purana JS bug/quirk hai, technically galat hai — legacy behavior hai jo fix nahi kiya gaya).

5. Strings

- **Concatenation (old way):** `'My name is ' + name + ' and I am ' + age`
- **Template literals (ES6, better way):** backticks (`` ` ``) use karke `${variableName}` syntax se variables directly string mein daal sakte ho.
- Common string properties/methods:
  - `.length` — characters ki ginti (property, parenthesis nahi lagti)
  - `.toUpperCase()` / `.toLowerCase()` — case change (method, parenthesis lagti hai)
  - `.substring(start, end)` — start index se end index (exclusive) tak substring
  - `.split(separator)` — string ko array mein todta hai given separator ke basis par (e.g. `','` se comma-separated values ko array bana sakte ho)
  - Methods ko **chain** kiya ja sakta hai, e.g. `.substring(0,5).toUpperCase()`

6. Arrays

- Array banane ke do tareeqe:
  - Constructor: `new Array(1,2,3)` (kam use hota hai)
  - Literal: `const fruits = ['apple', 'orange', 'pear']` (common way)
- JS arrays mein **alag-alag data types** ek sath rakhe ja sakte hain (statically typed nahi hai JS).
- **Zero-based indexing** — pehla element index 0 par hota hai.
- Element access: `fruits[1]`
- Common array methods:
  - `.push(value)` — end mein add
  - `.unshift(value)` — start mein add
  - `.pop()` — last element remove
  - `Array.isArray(value)` — check karta hai ke value array hai ya nahi (true/false)

    - `.indexOf(value)` — value ka index return karta hai
- **Higher order array methods** (functional programming style — inhe prefer kiya jata hai loops ke bajaye):
    - `.forEach(callback)` — har item par loop chalata hai, kuch return nahi karta
    - `.map(callback)` — naya array return karta hai (transform kiya hua)
    - `.filter(callback)` — naya array return karta hai jo condition match kare
    - In methods ko **chain** kar sakte ho, e.g. `.filter(...).map(...)`

7. Objects (Object Literals)

- Key-value pairs ka structure: `const person = { firstName: 'John', lastName: 'Doe', age: 30 }`
- Objects ke andar **arrays** aur **nested objects** bhi ho sakte hain.
- Access karne ka tareeqa: dot notation `person.firstName`
- Nested access: `person.address.city`
- **Destructuring** (ES6): `const { firstName, lastName } = person;` — object se values nikal ke seedha variables bana dete hain. Nested ke liye: `const { address: { city } } = person;`
- Property directly add karna: `person.email = 'john@gmail.com'`
- Array of objects: common pattern hai (e.g. todos list), inn par bhi array methods (`forEach`, `map`, `filter`) chalti hain.

8. JSON

- JSON (JavaScript Object Notation) — data format hai, APIs/backend ke sath data bhejne-lene ke liye use hota hai.
- Object literal jaisa hi hota hai, farq: **saari keys aur strings double quotes mein** honi chahiye (single quotes allowed nahi).
- `JSON.stringify(object)` — JS object ko JSON string mein convert karta hai (server ko bhejne ke liye).

9. Loops

- **for loop**: `for (let i = 0; i < 10; i++) { }` — teen parts: initialization, condition, increment.
- **while loop**: variable loop se bahar declare hota hai, phir `while(condition) { }`. Increment manually karna zaroori hai warna infinite loop ban jayega.
- **for...of loop**: arrays loop karne ka zyada readable tareeqa: `for (const item of array) { }`
- Higher order array methods (`forEach`, `map`, `filter`) generally for/while loops se **better** hain array iteration ke liye.

10. Conditionals

- **if / else if / else**: normal conditional structure.
- **Comparison operators**:
  - `==` (double equal) — sirf value compare karta hai, type nahi
  - `===` (triple equal) — value **aur** type dono compare karta hai (recommended, zyada predictable)
- **Logical operators**:
  - `||` (OR) — ek condition true ho to kaafi hai
  - `&&` (AND) — dono conditions true honi chahiye
  - In operators se nested if-statements avoid ki ja sakti hain.
- **Ternary operator**: shorthand if-else, variable assignment ke liye common: `const color = x > 10 ? 'red' : 'blue';`
- **switch statement**: jab multiple specific values check karni ho:

```
js
```

```
switch(color) {
  case 'red': /* ... */ break;
  case 'blue': /* ... */ break;
  default: /* ... */
}
```

Har case ke baad `break` lagana zaroori hai warna next case bhi chal jayega.

11. Functions

- Normal function: `function addNums(num1, num2) { return num1 + num2; }`
- **Default parameters:** `function addNums(num1 = 1, num2 = 1) { }` — agar argument na diya jaye to default value use hogi.
- Argument na dene par (bina default ke) result `NaN` (Not a Number) aata hai.
- **Arrow functions** (ES6): `const addNums = (num1, num2) => { return num1 + num2; }`
  - Single expression ho to curly braces aur `return` keyword drop kar sakte hain: `(num1, num2) => num1 + num2`
  - Sirf ek parameter ho to parenthesis bhi drop ho sakti hai: `num1 => num1 + 5`
  - Arrow functions ka ek concept hota hai **lexical** `this` — `this` ka scoping normal functions se different hota hai (advanced topic).

12. Object-Oriented Programming (OOP)

Constructor Functions (ES5 style)

- Capital letter se naming convention: `function Person(firstName, lastName, dob) { this.firstName = firstName; ... }`
- Instantiate karna: `const person1 = new Person('John', 'Doe', '4-3-1980');`
- `new Date('string')` se string ko actual **Date object** mein convert kar sakte hain, jispar methods (`.getFullYear()` waghera) chalte hain.

- Methods object mein directly add kar sakte hain (`this.getFullYear = function() {...}`), lekin better practice hai:
- **Prototype:** `Person.prototype.getFullName = function() {...}` — methods ko prototype par add karna better hai (har instance mein duplicate nahi hote, memory-efficient).

ES6 Classes (Syntactic Sugar)

- Classes **peeche se** prototypes hi use karti hain — bas likhne ka tareeqa zyada clean/readable hota hai.

```js
class Person {
  constructor(firstName, lastName, dob) {
    this.firstName = firstName;
    this.lastName = lastName;
    this.dob = dob;
  }
  getFullName() {
    return `${this.firstName} ${this.lastName}`;
  }
}
```

- Instantiation same tareeqe se: `new Person(...)`

13. DOM (Document Object Model)

- DOM = document ka tree structure (HTML tags waghera).
- **window object** browser ka top-level parent object hai (`alert`, `localStorage`, `fetch` sab isi mein hote hain). `window.` prefix zaroori nahi hota kyunke ye top level hai.
- **document object** DOM ko represent karta hai — selection ke liye use hota hai.

Single Element Selectors

- `document.getElementById('id')` — purana tareeqa, sirf ID se
- `document.querySelector('.class' | 'tag' | '#id')` — modern, versatile (jQuery jaisa), sirf **pehla match** return karta hai

Multiple Element Selectors

- `document.querySelectorAll('.item')` — **NodeList** return karta hai (array jaisa, array methods chal sakti hain jaise forEach) — **recommended**
- `document.getElementsByClassName()` / `getElementsByTagName()` — **HTMLCollection** return karte hain, direct array methods nahi chal sakti (manually convert karna parta hai) — kam use hote hain

DOM Manipulation

- `.remove()` — element hata dena
- `.lastElementChild` / `.firstElementChild` / `.children[index]` — child elements access karna
- `.textContent` / `.innerText` — text change karna
- `.innerHTML` — HTML dynamically insert karna
- `.style.propertyName = 'value'` — CSS property directly change karna (e.g. `.style.background = 'red'`)
- `.classList.add('className')` / `.classList.remove('className')` — class add/remove karna

Events

- `element.addEventListener('event', callback)` — e.g. `'click'`, `'submit'`, `'mouseover'`, `'mouseout'`
- Callback function ko **event object** milta hai (conventionally `e`).
- `e.preventDefault()` — default behavior rokta hai (jaise form submit hone par page reload/navigate hona)

- `e.target` — jis element par event fire hua uska reference deta hai (e.g. `e.target.className`)

14. Mini Project Pattern (Form + List)

- DOM se saare zaroori elements ko variables mein store karna (form, inputs, message div, list container).
- Form par `submit` event listener lagana, `preventDefault()` call karna.
- **Validation**: input `.value === ''` check karke error message dikhana.
- `setTimeout(callback, milliseconds)` — kisi cheez ko delay ke baad execute/remove karne ke liye (e.g. 3 second baad error message hatana).
- `document.createElement('li')` — naya element create karna
- `.appendChild(document.createTextNode('text'))` — text node ko element mein add karna
- `parentElement.appendChild(newElement)` — naye element ko DOM mein insert karna
- Submit ke baad input fields clear karna: `input.value = ''`

15. Beyond This Crash Course

- Data ko **persist** (permanently save) karne ke liye:
  - Backend + database (Node.js, PHP, Python waghera) + **fetch API** / **Ajax** se requests bhejna
  - **localStorage** — browser mein hi data store karna (sirf us user ke browser tak mehdood)

16. Objects — Deep Dive

- Real-life object ki tarah, JS object ke bhi **properties** (cheezein jo usme hain) aur **methods** (cheezein jo woh kar sakta hai) hote hain.
  - Example: phone → properties: color, size, model; methods: ring, take a picture.

    - Example: user object → properties: email, username, age; methods: login, logout.
- JavaScript mein kuch **built-in objects** already hote hain, jaise `Math` object aur `Date` object.
- Apne khud ke objects banane ka ek tareeqa hai **object literal notation** (curly braces `{}` use karke) — jaise arrays ke liye square brackets `[]` use hote hain.

Object Literal Banana

```js
const user = {
  name: 'Crystal',
  age: 30,
  email: 'crystal@ninjacode.uk',
  location: 'Berlin',
  blogs: ['Why mac-and-cheese rules', 'Ten things to make with Marmite']
};
```

- Har property ek **key-value pair** hoti hai.
- Readability ke liye har property **naye line** par likhna better hai (ek line mein bhi likh sakte ho, functionally same hai).
- Aakhri property ke baad **comma nahi lagta**, baaki sab ke baad lagta hai.
- Object ko console mein log karne par `__proto__` property bhi dikhti hai — abhi cover nahi karna, OOP wale chapter mein aayega.

Properties Access & Update Karna

- **Dot notation:** `user.name` → value milti hai. Update: `user.age = 35;`
- **Square bracket notation:** `user['name']` — key **string format** mein deni hoti hai.
    - Square bracket notation ka faida ye hai ke **variable** ke through key pass kar sakte hain:

```js
const key = 'location';
user[key]; // same as user.location
```

- Dot notation mein `user.key` likhne se ye literally "key" naam ki property dhoondega, variable ki value nahi lega — isliye dynamic key ke liye square brackets zaroori hain.
- Zyadatar cases mein **dot notation** prefer kiya jata hai, square bracket sirf dynamic keys ke liye.
- `typeof user` → `"object"` return karega.

Object Mein Methods Add Karna

- Method bhi ek key-value pair hi hoti hai, bas value ek **function** hoti hai:

```js
const user = {
  name: 'Crystal',
  login: function() {
    console.log('User logged in');
  }
};
user.login(); // method call karna
```

- Ye bilkul waisa hi hai jaise string methods (`.toUpperCase()`) kaam karte hain — value pe dot, method name, phir parentheses se invoke.
- Farq sirf itna hai ke normal function kisi object ke andar define nahi hoti, method **object ke andar** define hoti hai.

`this` Keyword

- Agar method ke andar object ki apni property (jaise `blogs`) access karni ho, to seedha `blogs` likhne se error aata hai ("blogs is not defined").
- Iske liye **`this` keyword** use karte hain: `this.blogs`.
- `this` ek **context object** hai — jis context mein code chal raha hai usko represent karta hai:
  - Agar document ke root (global scope) mein `this` log karo, to **`window` object** milta hai (global context).
  - Agar `this` kisi **method ke andar (regular function se banaya gaya)** use karo aur method ko object ke through call karo, to `this` us **object** ko refer karta hai jispar method call hua.
- **Important rule**: Jab method ko call karte hain, JS `this` ki value automatically us object pe set kar deta hai jispar method use hua.

Arrow Functions aur `this` ka Farq

- **Regular function** methods ke andar `this` ko us object se bind karta hai jispar method call hua.
- **Arrow function** methods ke andar `this` ko **bind nahi karti** — arrow function `this` ki value change nahi karti, wahi value use karti hai jo **surrounding/current context** mein already thi (yahan global `window` object).
- Isi wajah se agar method arrow function se banayi jaye, to `this.blogs` kaam nahi karega (kyunke `this` window object ko refer karega, jispar `blogs` property exist nahi karti).
- **Rule of thumb**: Object ke andar methods banate waqt **regular function use karo**, arrow function nahi, taake `this` sahi object ko refer kare.

Method Shorthand Syntax (ES6)

- Method likhne ka aur bhi chota tareeqa hai — `function` keyword aur colon hata ke seedha parentheses lagana:

```js
const user = {
  logBlogs() {
    console.log(this.blogs);
  }
};
```

- Ye ab bhi ek **regular function** hai (arrow function nahi), isliye `this` normal tareeqe se hi kaam karta hai. Ye syntax future mein methods banane ka preferred tareeqa hai.

Array Ke Andar `forEach` se Loop

```js
logBlogs() {
  console.log('This user has written the following blogs:');
  this.blogs.forEach(blog => {
    console.log(blog);
  });
}
```

- `this.blogs` ek array hai, isliye `.forEach()` method use ho sakta hai.
- `forEach` ke andar arrow function use karna theek hai kyunke yahan `this` ki zaroorat nahi (sirf `blog` parameter use ho raha hai).

17. Array of Objects

- Agar array ke har element ke paas **multiple properties** honi chahiye (jaise blog ka title, likes, content, author), to sirf string store karne ke bajaye har element ko **object** banate hain:

```js
```

```
const blogs = [
  { title: 'Why mac-and-cheese rules', likes: 30 },
  { title: 'Ten things to make with Marmite', likes: 50 }
];
```

- Ye pattern bohot common hai — real-world data (jaise database se aane wala data) usually **array of objects** ke format mein hi hota hai (e.g. movies collection, jahan har movie ek object hai).
- Loop karte waqt har item object hoti hai, is liye uski properties access ki ja sakti hain:

```
js

blogs.forEach(blog => {
  console.log(blog.title, blog.likes);
});
```

18. Math Object (Built-in Object)

- `Math` object (capital M) JavaScript ka built-in object hai jo ready-made **properties aur methods** deta hai.
- Properties (constants):
  - `Math.PI` — 3.14159... (pi ki value)
  - `Math.E` — Euler's number
- Common methods:
  - `Math.round(num)` — nearest integer tak round karta hai
  - `Math.floor(num)` — hamesha **neeche** wale integer tak round karta hai
  - `Math.ceil(num)` — hamesha **upar** wale integer tak round karta hai
  - `Math.trunc(num)` — sirf decimal part hata deta hai (round nahi karta, bas cut karta hai)

- `Math.random()` — **0 aur 1 ke beech** ek random decimal number deta hai
  - 1 se 100 ke beech random number chahiye ho to: `Math.round(Math.random() * 100)`

19. Primitive vs Reference Types

- JavaScript mein **7 data types** hain, jinme se 6 **primitive** hain: `number`, `string`, `boolean`, `null`, `undefined`, `symbol`.
- **Reference type** sirf ek category hai: **object** (isme object literals, arrays, functions, dates, aur baaki sab built-in objects jaise `Math` shaamil hain).
- Farq **memory storage** ka hai:
  - **Primitive values** → **stack** memory mein store hoti hain (chhoti, fast access, limited space).
  - **Reference types (objects)** → **heap** memory mein store hote hain (zyada space, thoda slow), aur stack par sirf ek **pointer** (jo heap wali location ko point karta hai) store hota hai.

Primitive Types Copy Karna

```js
let score1 = 50;
let score2 = score1; // NAYI, ALAG value stack par copy hoti hai
score1 = 100;
// score1 = 100, score2 = 50 (dono independent hain)
```

- Jab primitive value copy karte hain, to stack par **naya, alag value** bun jata hai. Ek ko change karne se doosra affect nahi hota.

Reference Types Copy Karna

```js
```

```
const user1 = { name: 'Ryu', age: 30 };
const user2 = user1; // sirf POINTER copy hota hai, object copy nahi hota
user1.age = 40;
// user1.age = 40, user2.age bhi 40 (dono same object ko point karte hain)
```

- Jab object copy karte hain, to sirf **pointer copy** hota hai, actual object heap par **ek hi baar** store rehta hai.
- Dono variables (`user1`, `user2`) **same object** ko point karte hain — isliye ek se change karne par doosre mein bhi change nazar aata hai.
- Ye JS mein ek bohot common **gotcha/confusion point** hai — object copy karte waqt hamesha yaad rakhna ke ye sirf reference (pointer) copy hota hai, naya independent object nahi banta.

20. Object-Oriented Programming (OOP) — Fundamentals

- OOP ek **programming paradigm** hai (style of programming), koi language ya tool nahi. Ye 1970s se hai lekin aaj bhi relevant hai. Kai languages ye support karti hain: C#, Java, Ruby, Python, JavaScript.
- OOP se pehle **procedural programming** hoti thi — data variables mein separate, aur functions unpar operate karte the (parameters ke through). Jaise-jaise program bara hota hai, ye **"spaghetti code"** ban jata hai — functions ek doosre se tightly interdependent ho jate hain, ek change karne se doosre break ho jate hain.
- OOP mein related variables (**properties**) aur functions (**methods**) ko ek unit — **object** — mein group kar dete hain. Isse related data aur logic ek jagah rehta hai.
- Procedural function zyada **parameters** leta hai (kyunke saari data variables se pass karni parti hai). OOP mein wahi data object ki properties ban jati hain, isliye method ke parameters kam ho jate hain — kam parameters = easier to use/maintain function.

Four Core Pillars of OOP

1. **Encapsulation** — related variables aur functions ko ek object mein group karna. Faida: complexity kam hoti hai, aur object ko reuse kiya ja sakta hai.
2. **Abstraction** — complex internal details **hide** karna, sirf essential/public interface expose karna (jaise DVD player — andar complex circuit hai, bahar sirf kuch buttons). Faida: interface simple ho jata hai, aur agar internal implementation change ho, to bahar wale code par asar nahi parta (impact of change kam hota hai).
3. **Inheritance** — redundant/duplicate code khatam karne ka mechanism. Common properties/methods ek generic object mein define karke, doosre objects unhe **inherit** kar sakte hain (jaise HTML elements — sab mein `hidden`, `innerHTML`, `click`, `focus` common hain, ye ek base object mein define karke baaki elements inherit karte hain).
4. **Polymorphism** — "poly" = many, "morph" = form → "many forms". Ye lambi `if/else` ya `switch/case` chains ko khatam karta hai. Har object apna khud ka method (e.g. `render()`) implement karta hai jo uske type ke hisaab se alag behave karta hai — is se ek hi line of code se sab objects handle ho jate hain, switch-case ki zaroorat nahi rehti.

21. Objects Banane Ke Tareeqe (Factory vs Constructor)

Object Literal Ka Masla

- Agar object mein **methods (behavior)** hain aur usi jaisa doosra object bhi chahiye, to object literal syntax se **code duplicate** karna parta hai — agar method mein bug ho, to har jagah fix karna parega.
- Simple data-only object (bina methods ke) duplicate karne mein koi masla nahi.

Factory Function

- Ek regular function jo object **return** karta hai:

```
js
```

```
function createCircle(radius) {
  return {
    radius,          // ES6 shorthand: key aur value ka naam same ho to sirf
    draw() {
      console.log('draw');
    }
  };
}
const circle = createCircle(1);
```

Constructor Function

- Naming convention: **pehla letter capital** (jaise `Circle`). Ye class jaisi lagti hai lekin technically ek regular function hi hai — JavaScript mein (ES6 classes se pehle) classes ka concept nahi tha.

```
js

function Circle(radius) {
  this.radius = radius;
  this.draw = function() {
    console.log('draw');
  };
}
const circle = new Circle(1);
```

- `this` keyword us object ko refer karta hai jo currently code execute kar raha hai.

`new` Operator Ke Peeche Kya Hota Hai

Jab `new` operator se function call karte hain, teen cheezein hoti hain:

1. Ek **naya empty object** create hota hai.

2. `this` ko us naye object ki taraf **point** kar diya jata hai (agar `new` use na karein, to `this` by default **global object** — browser mein `window`, Node mein `global` — ko point karta hai; isliye `new` bhoolna khatarnak hai, kyunke properties global object par ban jati hain).
3. Wo naya object function se **automatically return** ho jata hai (explicit `return` likhne ki zaroorat nahi).

Factory vs Constructor — Kaunsa Use Karein?

- Dono regular functions hain — bas **pattern** ka farq hai (agar object return karte hain → factory function; agar `this` + `new` use karte hain → constructor function).
- C#/Java background wale developers usually constructor function prefer karte hain (class-like feel deta hai). Kuch developers factory function prefer karte hain (`new` bhoolne ka risk avoid karne ke liye — lekin modern JS mein `new` bhoolne par error aa jata hai).
- Dono patterns real projects mein milte hain, dono samajhna zaroori hai.

`constructor` Property

- Har JS object mein ek `constructor` property hoti hai jo us function ko reference karti hai jisne wo object banaya.
- `circle.constructor` → `Circle` function dikhayega.
- Object literal (`{}`) internally JS engine ke through `new Object()` constructor se banta hai — isliye har object literal ka bhi `constructor` property built-in `Object` function ko point karti hai.
- Baaki built-in constructors: `String`, `Boolean`, `Number` — lekin inke bajaye **literals** (`'text'`, `true`, `123`) use karna cleaner hota hai.

22. Functions Bhi Objects Hain

- JavaScript mein **functions bhi objects** hote hain — inke bhi apni properties aur methods hoti hain (e.g. `functionName.name`, `functionName.length`).

- Har function ka `constructor` bhi ek built-in `Function` constructor ko refer karta hai.
- Functions par kuch important methods:
  - `.call(context, arg1, arg2, ...)` — function ko call karta hai, pehla argument `this` ki value set karta hai, baaki arguments explicitly diye jate hain.
  - `.apply(context, [args])` — `.call()` jaisa hi hai, farq sirf itna ke arguments ek **array** ke through pass hote hain.
  - Jab `new` operator use karte hain, internally ye `.call()` jaisa hi mechanism use karta hai — empty object create karke usko `this` ke first argument ki jagah pass karta hai.

23. Primitive vs Reference — Function Arguments Ke Sath

- Jab function ko **primitive value** (number, string, boolean) pass karte hain, to us variable ki **value copy** ho ke function ke parameter mein jati hai — dono independent ho jate hain. Function ke andar value change karne se **bahar wali original value par koi asar nahi parta**.
- Jab function ko **object (reference type)** pass karte hain, to us object ka **reference/pointer copy** hota hai — parameter aur original variable **same object** ko point karte hain. Function ke andar object ki property change karne se **bahar wala original object bhi change ho jata hai**.
- Yaad rakhne wali baat: **primitives value ke through copy hote hain, objects reference ke through.**

24. Object Properties — Dynamic Nature

- JS objects **dynamic** hote hain — create karne ke baad bhi properties **add** ya **delete** ki ja sakti hain (classes nahi hain, isliye structure pehle se fix karne ki zaroorat nahi). Ye real-world use case mein bohot useful hai (jaise server par user object mein baad mein `token` property add karna).
- Property add karna: `obj.newProp = value;`

- Property delete karna: `delete obj.propName;`
- **Bracket notation** ka use dynamic key access ke liye hota hai (jab property ka naam ek variable mein store ho):

```js
const propertyName = 'location';
circle[propertyName]; // dot notation se ye possible nahi (circle.propertyNam
```

- Bracket notation zaroori hoti hai jab property name mein **special characters ya space** ho (jaise `'center-location'`), jo valid identifier nahi ban sakta.

Object Enumeration (Properties/Methods Ginte Hue Loop Karna)

- `for...in` **loop**: object ki saari keys (properties + methods) par loop karta hai:

```js
for (let key in circle) {
  console.log(key);         // key ka naam
  console.log(circle[key]); // uski value (bracket notation zaroori)
}
```

- `typeof` operator se properties aur methods alag kiye ja sakte hain (`typeof circle[key] !== 'function'`).
- `Object.keys(obj)` — saari keys ka **array** return karta hai (properties + methods dono, alag nahi kar sakte).
- `in` **operator** — check karta hai ke koi property/method object mein exist karti hai ya nahi: `'radius' in circle` → `true`/`false`.

25. Abstraction Implement Karna — Private Members & Closures

- Kisi property/method ko object se **hide** karna ho (implementation detail), to usse object ki property (`this.x`) banane ke bajaye constructor function ke andar sirf ek

local variable define karte hain:

```js
function Circle(radius) {
  this.radius = radius;
  let defaultLocation = { x: 0, y: 0 }; // ye object ke bahar accessible NAHI
  let compute = function() { /* ... */ }; // ye bhi private hai
  this.draw = function() {
    compute(); // andar se access ho sakta hai
    console.log(this.radius);
  };
}
```

- Ye local variables technically object ki properties nahi hain, isliye bahar se access nahi ho sakti — lekin OOP perspective se inhe object ke **"private members"** kaha jata hai.

Closure Ka Concept

- **Scope** temporary hoti hai — function khatam hone par uske local variables **die** ho jate hain.
- **Closure** ye determine karti hai ke ek inner function apne parent function ke variables ko access kar sake, chahe parent function pehle hi execute ho chuka ho.
- Jab hum ek function ke andar doosra function banate hain (jaise upar `draw` method `Circle` function ke andar hai), to inner function apne parent ke saare local variables **"remember"** kar leta hai — is memory ko closure kehte hain.
- Isi wajah se `draw` method andar se `defaultLocation` aur `compute` (jo `Circle` function ke local/private variables hain) access kar pata hai, jab ke bahar se koi access nahi kar sakta.

Getters aur Setters (`Object.defineProperty`)

- Agar kisi private variable ko **read-only** tareeqe se access karwana ho (bahar se sirf padh sakein, set na kar sakein), to `Object.defineProperty()` use karte hain:

```js
Object.defineProperty(this, 'defaultLocation', {
  get: function() {
    return defaultLocation; // closure se access
  },
  set: function(value) {
    if (!value.x || !value.y) {
      throw new Error('Invalid location');
    }
    defaultLocation = value;
  }
});
```

- `get` — property padhne (read) ke liye function; property ko normal property ki tarah access kar sakte hain (`circle.defaultLocation`), method ki tarah call nahi karna parta.
- `set` — property set karne ke liye function; ismein **validation logic** likh sakte hain (invalid value par error throw kar sakte hain via `throw new Error('message')`).
- Agar sirf `get` define ho aur `set` nahi, to wo property **read-only** ban jati hai (set karne ki koshish par error aata hai).

26. Variable Naming Rules (Detailed)

- Variable naam mein sirf **letters (a-z, A-Z), numbers, underscore (`_`), aur dollar sign (`$`)** allowed hain. Koi aur special character (jaise `.`, round brackets) allowed nahi.

- Variable naam **number se start nahi ho sakta** — lekin number beech ya end mein ho sakta hai (e.g. `username1` valid hai).
- **Reserved keywords** (jaise `var`, `let`, `function`, `for`) variable naam ke tor par use nahi ho sakte.
- Variables **case-sensitive** hote hain — `myage` aur `myAge` do alag variables hain.
- Naming conventions (multi-word variable naam ke liye):
  - **camelCase** — pehla word chota, baaki words ka pehla letter capital (e.g. `userName`) — JavaScript mein ye standard convention hai.
  - **snake_case** — words ke beech underscore (e.g. `user_name`).
- Variable naam **descriptive/intuitive** hone chahiye (jo data represent kar rahe hain uska matlab clear ho) — single letters (`a`, `b`) ya overly long names avoid karein.
- Strings likhte waqt agar text ke andar hi apostrophe ho (jaise `Naveen's phone`), to us string ko **double quotes** mein likhna better hai, taake single quote confusion na ho (ya phir apostrophe ko escape karna parta).

27. Scope — Global, Function, Aur Block

- **Global scope**: jo variable kisi function ya block ke bahar declare ho, wo **har jagah** available hoti hai — chahe `var`, `let`, ya `const` se banaya ho, teeno global scope mein same tarah behave karte hain.
- **Local scope** do tarah ki hoti hai:
  - **Function scope** — function ke andar declare kiya gaya variable sirf usi function ke andar accessible hota hai.
  - **Block scope** — curly braces `{ }` ke andar declare kiya gaya variable (`if`, `for`, `while`, `switch`, ya sirf `{ }`) sirf usi block ke andar accessible hota hai.
- **Values sirf neeche se upar (child se parent) ki taraf mil sakti hain, upar se neeche nahi jaa sakti** — matlab agar variable global scope mein hai, to function/block ke andar accessible hoga, lekin agar variable sirf block ke andar declare hua hai, to wo block ke bahar (parent function ya global scope mein) accessible **nahi** hoga.

- Jab koi variable kisi scope mein nahi milta, JS engine **upar wale (parent) scope** mein dhoondta hai — block se function tak, function se global tak.

`var` ka Alag Behavior (Function-Scoped, Block-Scoped Nahi)

- `var` **sirf function-scoped hota hai, block-scoped nahi** — matlab agar `var` ko kisi `if` block ya `for` loop ke andar declare karein, to uski value **block ke bahar (jis function ke andar wo block hai) bhi accessible/modifiable** hoti hai — `var` curly braces ko "ignore" kar deta hai.
- `let` **aur** `const` **dono block-scoped hain** — curly braces ke andar declare ki gayi value sirf usi block tak mehdood rehti hai, bahar leak nahi hoti.
- **Redeclaration:** `var` se same naam ka variable **dobara declare** karna allowed hai (error nahi aata, sirf overwrite ho jata hai — ye confusing aur bug-prone hai, especially bade projects/teams mein). `let` aur `const` se same scope mein dobara declare karne par **error** aata hai ("Identifier has already been declared") — ye behavior **desirable** hai kyunke early error catch hoti hai.
- **Reassignment:** `var` aur `let` dono reassignment allow karte hain (`x = newValue`, bina keyword repeat kiye). `const` reassignment allow **nahi** karta.
- **General recommendation:** `var` ko avoid karo (sirf legacy code mein milega), jab tak reassign nahi karna to `const` use karo, jab reassign karna ho to `let` use karo.

28. Loops — Detailed (for, while, do-while, for-in, for-of)

JavaScript mein 5 tarah ke loops hote hain: **for**, **while**, **do-while**, **for-in**, aur **for-of**.

`for` Loop

```js
for (let i = 0; i < 10; i++) {
  console.log(i);
}
```

- Teen parts: **initialization** (`let i = 0`), **condition** (`i < 10`), **increment/decrement expression** (`i++`).
- Increment sirf `1` hi nahi, koi bhi step size ho sakta hai (e.g. `i = i + 5`, ya table banane ke liye).
- Loop **reverse** order mein bhi chala sakte hain: initialization high value se start karo, condition `i > 0` rakho, aur `i--` (decrement) use karo.
- Odd/even numbers filter karne ke liye `if (i % 2 === 0)` jaisi condition andar laga sakte hain.

`while` Loop

```js
let i = 0;
while (i < 10) {
  console.log(i);
  i++;
}
```

- Variable **loop se bahar** initialize hota hai, phir sirf condition check hoti hai.
- **Important**: agar increment/decrement bhool jayein, to loop kabhi khatam nahi hoga — ye **infinite loop** ban jata hai jo application/browser ko **crash/freeze** kar sakta hai.

`do-while` Loop

```js
let i = 0;
do {
  console.log(i);
  i++;
} while (i < 10);
```

- `while` jaisa hi hai, farq ye hai ke condition **code block ke baad** check hoti hai.
- Isi wajah se **guaranteed hai ke block kam se kam ek baar zaroor chalega**, chahe condition shuru se hi false ho (jaise `i = 0` aur condition `i < 0` ho, tab bhi ek baar chalega, phir exit karega).
- `do-while` real-world coding mein **kam use hota hai** — bohot rare cases mein zaroorat parti hai.

`for...in` Loop

```js
const animal = { name: 'Dog', leg: 4 };
for (let key in animal) {
  console.log(key);         // sirf keys milti hain: 'name', 'leg'
  console.log(animal[key]); // value ke liye bracket notation zaroori
}
```

- **Objects** par loop karne ke liye use hota hai — ye object ki har **key** (property name) deta hai.
- Value nikalne ke liye **bracket notation** use karni parti hai (`obj[key]`), dot notation se possible nahi kyunke key ek variable hai.
- Arrays par bhi use ho sakta hai, lekin tab ye **index numbers** deta hai (0, 1, 2, ...), values nahi.

`for...of` Loop

```js
const names = ['Rahul', 'Neha', 'Aman'];
for (let name of names) {
  console.log(name); // seedha value milti hai, index nahi
}
```

- **Arrays** (aur doosre iterable structures) par loop karne ke liye use hota hai — ye seedha **value** deta hai, index nahi.
- `for...in` array-index deta hai, `for...of` array-value deta hai — ye dono ka main farq hai.

29. Primitive Data Types — Immutability (MDN)

- JavaScript mein **7 primitive data types** hain: `string`, `number`, `bigint`, `boolean`, `undefined`, `symbol`, `null`.
- Primitive value **object nahi hoti** aur uski apni koi property ya method nahi hoti.
- **Primitives immutable hote hain** — matlab unhe **change/alter nahi kiya ja sakta**. Jab hum ek variable ko naya value dete hain (reassign), to purani value change nahi hoti — sirf variable ek naye value ko point karne lag jata hai.
- Farq samajhna zaroori hai: **variable reassign** ho sakta hai, lekin **primitive value khud mutate nahi hoti** — ye objects/arrays/functions se alag hai jo mutate ho sakte hain.
- **Auto-boxing**: Primitives ke paas khud methods nahi hote, lekin jab hum `"hello".toUpperCase()` jaisa method call karte hain, JavaScript automatically ek temporary **wrapper object** bana deta hai (e.g. `String` wrapper), method us wrapper object par call hoti hai, aur result return hone ke baad wrapper object discard ho jata hai. Isi wajah se `str.customProp = 1;` jaisa kuch likhne se property save nahi hoti — kyunke wo ek temporary wrapper object par set ho rahi hoti hai, asal string par nahi.

30. Loops — Aur Deep (break, continue, labeled statements)

- `break` statement: kisi loop (ya `switch`) ko **turant terminate** kar deta hai, aur uske baad wale code par control chala jata hai.

```
js
```

```
for (let i = 0; i < arr.length; i++) {
  if (arr[i] === theValue) break; // match milte hi loop ruk jayega
}
```

- `continue` statement: current iteration **skip** kar deta hai (loop ko poora terminate nahi karta), aur agli iteration se loop continue hota hai.
  - `while` loop mein `continue` seedha condition check pe wapas jata hai.
  - `for` loop mein `continue` increment expression pe jata hai.
- **Labeled statements**: kisi loop ko ek **naam (label)** de sakte hain, taake nested loops mein `break`/`continue` specific loop ko target kar sakein:

js

```
outerLoop: while (true) {
  while (true) {
    if (someCondition) break outerLoop; // dono loops ek sath terminate
    else break; // sirf inner loop terminate
  }
}
```

- `for...in` **arrays** ke liye recommend nahi kiya jata — kyunke agar array mein koi custom property add ki ho, to `for...in` wo bhi return kar dega (sirf numeric indexes nahi). Arrays ke liye traditional `for` loop ya `for...of` behtar hain.
- `for...of` ke sath **destructuring** bhi use ho sakti hai, jaise object ke key-value pairs ek sath loop karne ke liye:

js

```
const obj = { foo: 1, bar: 2 };
for (const [key, val] of Object.entries(obj)) {
  console.log(key, val);
}
```

31. Conditionals — Aur Deep

Truthy / Falsy Values

- Conditional test mein sirf `true`/`false` hi nahi, **koi bhi value** test ho sakti hai — JS usse automatically boolean mein convert karta hai.
- **Sirf ye values `false` (falsy) treat hoti hain:** `false`, `undefined`, `null`, `0`, `NaN`, empty string (`''`). Baaki **sab kuch truthy** hota hai (non-empty strings, non-zero numbers, objects, arrays — chahe empty array/object bhi ho).
- Isi wajah se sirf variable name likh ke check kar sakte hain ke wo "exist" karta hai ya nahi:

```
js

if (cheese) { /* cheese ki value truthy hai */ }
```

`if / else if / else` — Nesting Aur Common Mistake

- `if` statements ko **nest** kiya ja sakta hai (ek `if` doosre `if` ke andar) — har ek independently apni condition check karta hai.
- Logical OR (`||`) use karte waqt ek common galti: `if (x === 5 || 7 || 10)` — ye **hamesha true** hoga, kyunke `7` khud hi truthy hai (JS `7` ko as-is boolean test kar raha hai, `x === 7` nahi kar raha). Sahi tareeqa: `if (x === 5 || x === 7 || x === 10)` — har condition **poori** likhni parti hai.

`switch` Statement

- Jab bohot saare specific values check karne hon (aur har case mein zyada complex logic na ho), to `switch` `if/else` chain se **cleaner** hota hai.

- Syntax: `switch(expression) { case value: /* code */ break; default: /* code */ }`
- **`break` bhoolna ek common bug hai** — agar `break` na lagayein, to matching case ke baad wale saare cases bhi chalte rahenge ("fall-through" behavior), jab tak koi `break` na mile.
- `default` case optional hai — agar expression kisi bhi case se match na ho, aur koi "unknown" value expect ho sakti ho, to `default` include karna chahiye.

Ternary Operator — Aur Use Cases

- Sirf variable assignment ke liye nahi, **koi bhi expression/function call** ternary ke andar chal sakta hai:

```js
select.value === 'black' ? update('black', 'white') : update('white', 'blac
```

- Syntax: `condition ? valueIfTrue : valueIfFalse`

32. Arrays — Deep Dive (Methods & Internals)

Arrays Technically Objects Hain

- JavaScript mein sirf **8 basic data types** hain, aur array in mein se ek nahi hai — **array ek special type ka object hai.** `arr[0]` bracket syntax wahi hai jo object property access (`obj[key]`) mein use hoti hai, bas yahan numbers keys hoti hain.
- Isi wajah se array **reference type** hai — copy karne par sirf reference copy hota hai (jaisa objects section mein cover kiya).
- Array engine internally memory mein elements ko **contiguous (ek doosre ke sath connected)** rakhta hai — isse special performance optimizations milti hain. Lekin agar array ko "galat tareeqe" se use karein (jaise `arr.customProp = 5`, ya `arr[99999] = 'x'` jab array chhota ho, ya reverse order mein fill karna), to ye optimizations

disable ho jati hain aur array slow ho jata hai. **Arrays ko sirf ordered data ke liye use karo**, arbitrary keys ke liye plain object use karo.

`length` Property Ka Special Behavior

- `length` actual element count nahi, balke **sabse bara numeric index + 1** hota hai.
- `length` ko manually **chota karna array ko truncate** kar deta hai (permanently, values wapas nahi aatin agar length dobara barhayein).
- Array clear karne ka simplest tareeqa: `arr.length = 0;`

Stack vs Queue (push/pop vs shift/unshift)

- Array ko **stack** ki tarah use kar sakte hain: `push()` (end mein add) + `pop()` (end se remove) — ise **LIFO** (Last-In-First-Out) kehte hain.
- Array ko **queue** ki tarah use kar sakte hain: `push()` (end mein add) + `shift()` (start se remove) — ise **FIFO** (First-In-First-Out) kehte hain.
- **Performance:** `push`/`pop` **fast** hote hain (kyunke sirf end pe kaam hota hai, baaki elements ko renumber nahi karna parta). `shift`/`unshift` **slow** hote hain (kyunke baaki saare elements ko re-index karna parta hai — index shift karna parta hai).

`splice()` — Remove/Insert By Index

```js
array.splice(startIndex, deleteCount, item1, item2, ...);
```

- Pehla argument: kahan se start karna hai. Doosra argument: kitne items delete karne hain. Uske baad optional naye items insert karne ke liye.
- E.g. `arr.splice(4, 1)` — index 4 se 1 item delete karega.

`slice()` — Array Copy Karna

```js
array.slice(startIndex, numberOfItemsOrEndIndex);
```

- Original array ko **change nahi karta**, ek **naya array** return karta hai.
- Bina arguments diye poora array copy ho jata hai.
- **Important**: agar array mein **objects** hain, to `slice()` sirf **references copy karta hai** — dono arrays same objects ko point karenge (ek array ke object ki property change karne se doosre array mein bhi wo change dikhega).

`at()` Method (Modern) — Negative Indexing

- JS mein negative index (`arr[-1]`) direct kaam nahi karta (`undefined` deta hai, kyunke bracket index ko literal treat karta hai).
- `arr.at(-1)` last element deta hai, `arr.at(-2)` second-last, waghera — ye modern aur cleaner tareeqa hai negative indexing ka.

Iteration Ke Multiple Tareeqe

- Traditional `for` loop (index ke through) — fastest, sab browsers mein kaam karta hai.
- `while` loop — manual index management, careful rehna parta hai warna infinite loop.
- `.forEach(callback)` — functional approach, callback ko `(item, index, array)` teen parameters milte hain (zyadatar sirf `item` use hota hai). `this` object ke andar bind nahi hoti custom tareeqe se agar arrow function use karein.
- `for...of` — seedha value deta hai, index nahi (agar index chahiye ho to `array.entries()` use karna parta hai jo `[index, value]` pairs deta hai — thora complex hai, isliye index chahiye ho to `.forEach()` ya traditional `for` behtar hai).
- `for...in` **arrays ke liye avoid karo** — property names (indexes as strings) deta hai, aur agar array mein custom properties hon to wo bhi include ho jati hain.

Sorting

- `.sort()` — array ko **ascending order** mein sort karta hai (default). Ye **destructive** method hai — original array ko **directly modify** karta hai (copy nahi banata).

- Descending order ke liye: `.sort().reverse()` — ya better performance ke liye custom **comparison function** pass karo:

```js
array.sort((a, b) => {
  if (a === b) return 0;
  return a > b ? -1 : 1; // descending order
});
```

- Comparison function `0` return kare to order same rehta hai, negative return kare to `a` pehle aata hai, positive return kare to `b` pehle aata hai.
- Objects ke array ko kisi specific property (jaise `cost` ya `text`) ke basis par sort karne ke liye, comparison function mein wahi property compare karo (`a.cost - b.cost` ascending ke liye common shorthand hai).

Searching Methods

- **`.indexOf(value)`** — value ka pehla index return karta hai, nahi mile to `-1`. Simple values (strings/numbers) ke liye best.
  - Optional second argument se **starting index** specify kar sakte hain (agar same value multiple baar ho aur baad wala occurrence chahiye ho).
- **`.lastIndexOf(value)`** — end se search karta hai (reverse direction), baaki `.indexOf()` jaisa hi.
- **`.indexOf()`**/**`.lastIndexOf()`** **objects ke liye kaam nahi karte** kyunke objects reference type hain — do "same dikhne wale" objects technically alag memory references hote hain.
- **`.findIndex(callback)`** — objects (ya kisi complex condition) ke liye index dhoondne ka tareeqa; callback `true`/`false` return karta hai, jo pehla item test pass kare uska index milta hai.
- **`.find(callback)`** — `.findIndex()` jaisa hi, lekin **index ke bajaye actual item** return karta hai. Match na mile to `undefined` return hota hai.

Existence Check Karna

- `.includes(value)` — simple values (strings/numbers) ke liye — value array mein hai ya nahi, `true`/`false` deta hai.
- `.some(callback)` — agar array ka **kam se kam ek** item condition pass kare to `true`.
- `.every(callback)` — agar array ke **saare** items condition pass karein tabhi `true`.

Merge Karna (Concatenation)

- `.concat(arr2, arr3, ...)` — multiple arrays ko ek **naye array** mein merge karta hai (original arrays change nahi hote), order preserve rehta hai.
- **Spread operator** se bhi same kaam ho sakta hai, declarative tareeqe se: `const merged = [...arr1, ...arr2, ...arr3];`

Array Ko String Mein Convert Karna

- `.join(separator)` — array ke elements ko ek string mein jodta hai, given separator se. Kuch na do to default comma (`,`) use hota hai.
  - E.g. `arr.join(', ')`, ya `arr.join('<br>')` HTML mein line breaks ke liye.
- Array ka **default** `toString()` bhi comma-separated string deta hai (`String([1,2,3])` → `'1,2,3'`).

Transform Karna: `map()` aur `filter()`

- `.map(callback)` — har element par callback chalata hai aur **naya array** return karta hai jisme transformed values hoti hain (original array change nahi hota).
- `.filter(callback)` — callback jo elements ke liye `true` return kare, sirf unhe include karke **naya array** return karta hai (original change nahi hota).
- Ye methods **chain** ki ja sakti hain — jaise pehle `.filter()` se sirf needed items nikalo, phir `.map()` se unhe HTML strings mein transform karo, phir `.join('')` se ek single string bana lo. Ye pattern "**method chaining**" kehlata hai aur bohot powerful/readable code deta hai.

Arrays Compare Karna

- Arrays ko `==` ya `===` **se directly compare mat karo** — dono arrays alag objects/references hain, chahe unke andar same values hi kyun na ho (`[] === []` hamesha `false` hota hai, chahe dono empty hi hon).
- Compare karne ka sahi tareeqa: loop (`for...of`) mein item-by-item compare karna, ya array methods (`every`) use karna.

33. Functions — Deep Dive

Parameter vs Argument

- **Parameter**: function declare karte waqt jo variable naam parentheses mein likhte hain (e.g. `function greet(name) {}` mein `name` parameter hai).
- **Argument**: function **call** karte waqt jo actual value pass karte hain (e.g. `greet('John')` mein `'John'` argument hai).
- Ye ek common confusion hai — bohot se developers dono ko interchange karte hain, lekin technically alag hain.
- Agar function ko required argument na diya jaye, to us parameter ki value `undefined` ban jati hai (default JS behavior).

`return` Statement — Important Gotchas

- Agar function kuch bhi `return` nahi karta, to uski **default return value** `undefined` hoti hai.
- `return` **aur jo value return karni hai, wo dono SAME LINE par honi chahiye.** Agar `return` likh ke naye line par value likhein, to JavaScript automatically **semicolon insert kar deta hai** `return` ke baad (ye "Automatic Semicolon Insertion / ASI" kehlata hai), jiski wajah se function `undefined` return kar dega aur neeche wala code kabhi nahi chalega:

```
js
```

```
function getValue() {
  return
    5; // yahan 'return;' ban jayega, 5 kabhi return nahi hoga
}
```

- Agar object return karna ho, to object ka **opening curly brace** `return` **ke sath hi same line par** hona chahiye.

Function Ke Andar Variable Scope (Recap + Emphasis)

- Function ke andar declare kiya gaya variable sirf **usi function ke andar accessible** hota hai — bahar se access nahi ho sakta.
- Function ke **bahar** declare kiya gaya variable, function ke **andar se accessible** hota hai (outer scope se inner scope milta hai, reverse nahi).
- Isi wajah se function ke andar variables banana ek tareeqa hai unhe "**private**" rakhne ka — bohot saari libraries isi pattern ko use karti hain.

Function Declaration vs Function Expression

- **Function Declaration:** `function greet() { ... }` — naam ke sath directly define hoti hai.
- **Function Expression**: function ko ek **variable mein store** karte hain, aur function khud **anonymous** (naam ke bagair) hoti hai:

```
js

const greet = function() {
  console.log('Hello');
};
```

- Dono ka behavior (call karne ka tareeqa) same hota hai (`greet()`), bas declare karne ka syntax alag hai.

Callback Functions

- Function ko **parameter ki tarah** doosre function mein pass kiya ja sakta hai — is pass kiye gaye function ko **callback function** kehte hain.
- Ye bohot powerful pattern hai — jo function callback receive karta hai, wo apni marzi se decide kar sakta hai ke us callback ko kab aur kaise call karna hai, aur callback ko custom data bhi pass kar sakta hai.

```js
function formatText(text, formatCB) {
  if (typeof formatCB === 'function') {
    return formatCB(text);
  }
  return text.toUpperCase(); // default behavior agar callback na diya ho
}
formatText('hello', function(text) {
  return text.toUpperCase()[0] + text.slice(1);
});
```

- `typeof` operator se check kar sakte hain ke pass kiya gaya parameter actually **function hai ya nahi** (`typeof param === 'function'`).

IIFE (Immediately Invoked Function Expression)

- Kabhi kabhi function ko **manually call** karne ki zaroorat nahi hoti — hum chahte hain ke function **create hote hi khud-b-khud chal jaye.** Iske liye function ko **parentheses mein wrap** karke turant call kar dete hain:

```js
(function() {
  console.log('Runs immediately');
})();
```

- Ye pattern **IIFE** kehlata hai.
- Bohot common use case: **library ka pura code IIFE ke andar rakhna** — taake library ke internal variables bahar se access na ho sakein (function scope ki wajah se privacy milti hai).

Arrow Functions — Syntax Steps

Traditional function expression se arrow function tak step-by-step simplification:

```js
// Traditional
const greet = function(a) { return a + 100; };

// Step 1: "function" keyword hatao, arrow lagao
const greet = (a) => { return a + 100; };

// Step 2: agar sirf ek expression return ho raha hai, curly braces aur "retu
const greet = (a) => a + 100;

// Step 3: agar sirf ek simple parameter hai, uske parentheses bhi hata sakte
const greet = a => a + 100;
```

- **Parentheses sirf tab hata sakte ho jab function ka sirf ek simple parameter ho.** Multiple parameters, zero parameters, ya default/destructured/rest parameters ho to parentheses **zaroori** hain.
- **Curly braces sirf tab hata sakte ho jab function body mein sirf ek expression directly return ho raha ho.** Agar body mein multiple statements hon, to curly braces aur explicit `return` zaroori hain.
- **Object literal return karne ka gotcha:** `() => { foo: 1 }` kaam nahi karega — JS ise ek function body (block) samjhega, object literal nahi (kyunke arrow ke baad

agar `{` ho to JS use statements block treat karta hai). Object return karna ho to **parentheses mein wrap** karo: `() => ({ foo: 1 })`.

Arrow Functions — Limitations (Bohot Important)

1. **Apni `this` binding nahi hoti** — arrow function `this` ki value **surrounding/lexical scope** se leti hai (jahan wo likhi gayi hai), function call hone ke tareeqe se nahi badalti. Isi wajah se arrow functions ko **object methods ke tor par use nahi karna chahiye** (jaisa objects section mein already cover kiya).
2. **Constructor ke tor par use nahi ho sakti** — `new` ke sath call karne par **TypeError** aata hai, aur inke paas `prototype` property bhi nahi hoti.
3. **`arguments` object nahi hota** — agar `arguments` use karna ho to arrow function ke bajaye rest parameters (`...args`) use karo, ya regular function use karo.
4. **Generator function nahi ban sakti** (`yield` keyword use nahi kar sakte).
5. **`call()`, `apply()`, `bind()` arrow function par asar nahi dalte** — kyunke arrow function ka `this` already lexically fix hota hai, ye methods use `this` ko override nahi kar sakte.

- **Class fields mein arrow function** (jaise `autoBoundMethod = () => {...}`) automatically instance ke `this` ko "capture" kar leti hai — is pattern ko **"auto-bound method"** kaha jata hai, aur ye event listeners/callbacks ke sath `this` ka masla khud-ba-khud solve kar deta hai (regular method ke sath `this.method = this.method.bind(this)` manually likhna parta, jo arrow function class field se automatic ho jata hai).
- `setTimeout()` ya event listeners ke andar arrow function use karna bohot common hai kyunke ye outer/surrounding `this` ko preserve karta hai (regular function use karne se `this` window/global object ban jata — jaisa objects section mein already dekha).

Naming Conventions (Best Practices)

- Function naam **action verb se start** hona chahiye — jaise `getUser`, `showUser`, `calculateArea`, `calculateDistance`. Ye compulsory nahi, lekin ek widely-followed

convention hai. Team ke andar sab ko isi convention par agree hona chahiye.

- **Single Responsibility principle**: ek function ka **sirf ek hi kaam** hona chahiye — jo uske naam se match kare. Agar `formatText` function hai, to usse sirf format karna chahiye, print ya kuch aur nahi. Agar multiple tasks chahiye, to unhe alag functions mein todo, aur unhe ek doosre ke andar se call karo.
- Function naam **descriptive** hone chahiye — naam padh ke pata chal jana chahiye ke function kya karta hai (`add` jaisa generic naam avoid karo, `calculateSum` ya `formatText` jaisa specific naam behtar hai).

34. `call()`, `apply()`, aur `bind()` — `this` Ko Control Karna

- Ye teeno **functions** par available methods hain (kyunke functions bhi objects hain — pehle bhi cover kiya).
- Teeno ka common maqsad: kisi function ke andar `this` **ki value ko manually set/override** karna.

`call(thisValue, arg1, arg2, ...)`

- Function ko **turant call** karta hai, pehla argument `this` ki value set karta hai, baaki arguments normal tareeqe se (comma-separated) diye jate hain.

```js
printName.call(user2); // user2 ke context mein printName run hoga
greet.call(user2, 'Hello', 'morning'); // arguments comma se
```

`apply(thisValue, [argsArray])`

- `call()` jaisa hi hai, farq sirf itna ke arguments ek **array** ke through pass hote hain, individually nahi.

```js
greet.apply(user2, ['Hello', 'morning']); // arguments array mein
```

`bind(thisValue, arg1, arg2, ...)`

- **Turant call nahi karta** — ye function ki **ek nayi copy return karta hai** jismein `this` (aur diye gaye arguments) already fix/bind ho chuke hote hain. Us returned function ko baad mein manually call karna parta hai.

```js
const boundGreet = greet.bind(user2); // naya function milta hai
boundGreet(); // ab isko call karo
```

- Agar `this` ki koi zaroorat na ho (jaise standalone utility function mein), to `call`/`apply`/`bind` mein pehla argument `null` pass kar sakte hain.

Function Borrowing

- Kisi doosre object ka method **"borrow"** karke apne object par use karna — `call()` ya `apply()` se possible hai:

```js
const user1 = { fName: 'A', lName: 'B', getFullName() { return `${this.fName}
const user2 = { fName: 'John', lName: 'Doe' }; // isme getFullName method nah
user1.getFullName.call(user2); // "John Doe" — user1 ka method borrow karke u
```

Function Currying (via `bind`)

- Function currying matlab: function ke kuch **parameters ko pehle se fix** kar dena, taake baad mein sirf baaki parameters dene parein.

```js
function multiply(a, b) { return a * b; }
const double = multiply.bind(null, 2); // 'a' hamesha 2 fix ho gaya
double(5); // 10 — yahan 5, 'b' ban gaya
```

- Real-world use case: agar multiple APIs ka **base URL alag** ho lekin logic same ho, to `bind()` se base URL ko pehle se fix kar sakte hain, aur baad mein sirf endpoint dena parega.

Event Listeners Mein `this` Ka Masla

- Jab kisi **class method** ko event listener ke andar reference karte hain (jaise `button.addEventListener('click', user.printName)`), to method ke andar `this` ki value **class ka object nahi**, balke wo **HTML element** ban jati hai jispar event fire hua — kyunke event listener callback ko us element ke context mein invoke karta hai.
- Iska fix: constructor ke andar method ko explicitly bind karna:

```js
constructor() {
  this.printName = this.printName.bind(this);
}
```

- Ya method ko **arrow function ke andar wrap karke** call karna: `addEventListener('click', () => user.printName())` — is se `this` object ke method call ki tarah normal tareeqe se resolve hota hai.
- Ye bug React class components mein bhi common tha — isi wajah se class components mein constructor ke andar `this.method = this.method.bind(this)` likhna parta tha.

35. Practice Exercises Reference (Objects & Arrays)

Ek GitHub challenge set mila jo objects/arrays methods (jo upar cover ho chuke hain) practice karne ke liye use ho sakta hai — `peeps` array aur `comments` object ke sath:

Easier (peeps array ke sath):

- Peeps ki total count nikalna
- Sab logon ke full names ka naya array banana

- Check karna ke sab 24 se bare hain (`.every()`)
- Check karna ke koi ek 26 se chota hai (`.some()`)
- 30 se kam age wale peeps ka `youngPeeps` array banana (`.filter()`)
- Peeps ko age ke hisaab se oldest-to-youngest sort karna (`.sort()`)
- Sirf first names ka `firstNamePeeps` array banana (`.map()`)

Harder (comments object aur peeps array ke sath):

- Saare comments ek `commentsArray` mein list karna
- "love" word wale comments ko `loveComments` mein filter karna
- Comments ko rating ke hisaab se sort karna (bina rating wale ignore karke)
- Comment id ko key aur text ko value banate hue ek naya `commentObj` banana
- Rating ke hisaab se comments group karke `groupedRatings` object banana
- Saare comments ki average rating nikalna
- Comments ko user ke hisaab se group karke `groupedPeepComments` banana

Ye exercises `.filter()`, `.map()`, `.sort()`, `.some()`, `.every()`, aur `reduce`-jaisi patterns ki practice ke liye achhe hain (jo is document mein array methods section mein cover ho chuke hain).

36. `this` Keyword — Deep Dive

- `this` ki value **kaise function call hua** usi se decide hoti hai, function kahan/kaise define hua usse nahi (**runtime binding**).
- **Regular function jab object ke method ki tarah call ho** (`obj.method()`): `this` us object ko refer karta hai jispar call hua — chahe wahi function kisi doosre object ko bhi assign kar diya jaye, `this` hamesha **call ke waqt wale object** ko refer karega, definition-time object ko nahi.
- **Standalone function call** (`func()`, bina kisi object ke prefix ke): non-strict mode mein `this` = global object (`window`); strict mode mein `this` = `undefined`.

- **Primitive value par method call** (jaise `(1).method()`): strict mode mein `this` primitive value hi rehti hai; non-strict mode mein JS use automatically wrapper object mein convert kar deta hai ("this substitution").
- `null`/`undefined` ko `this` set karne ki koshish karo, to non-strict mode mein `this` automatically `globalThis` ban jata hai.

Callbacks Mein `this`

- Jab function ko **callback** ki tarah pass karte hain (jaise array methods `.forEach()`), to `this` us API/method ke implementation par depend karta hai jo callback ko call kar raha hai — zyadatar cases mein `this` = `undefined` (non-strict) ya global object hoti hai, jab tak method khud koi `thisArg` parameter accept na kare (jaise `.forEach(callback, thisArg)`).

Arrow Functions Mein `this` (Recap + Detail)

- Arrow function apni `this` binding **create nahi karti** — jo bhi `this` uske **surrounding (lexical) scope** mein tha, wahi use karti hai — ise "closure over `this`" kehte hain.
- `call()`, `apply()`, `bind()` se arrow function ka `this` **override nahi ho sakta** — ye methods sirf arguments pass karne ke liye kaam karengi, `this` ignore ho jayega.

Constructors Mein `this`

- `new` ke sath function call hone par `this` naye banaye gaye object ko refer karta hai.
- Agar constructor function explicitly koi **object return** kare, to `this` binding discard ho jati hai aur wahi returned object result banta hai.

Class Context Mein `this`

- Class **instance methods** mein `this` us instance ko refer karta hai jispar method call hua.
- Class **static methods** mein `this` class ko khud refer karta hai (instance ko nahi).
- Class fields (instance ya static) bhi apne respective context (`this` = instance ya class) mein evaluate hote hain — isi wajah se class field mein arrow function likhne

se wo instance ke `this` ko "auto-bind" kar leti hai.

DOM Event Handlers Mein `this`

- Jab function ko event listener (`addEventListener`) ke through use karte hain, `this` us **DOM element** ko refer karta hai jispar listener attach hua hai.
- Class method ko event listener mein directly reference karne se `this` class instance ki jagah HTML element ban jata hai (already objects section mein cover ho chuka).

Class Methods "Bind" Karna

- Bohot common pattern: constructor ke andar `this.methodName = this.methodName.bind(this);` likhna, taake wo method kahin bhi (event listener ke andar bhi) call ho, `this` hamesha instance ko refer kare.
- Iska tradeoff: har instance apni khud ki bound method copy banati hai, jo thodi extra memory use karti hai — isliye sirf zaroorat honay par hi use karo.

37. Asynchronous JavaScript — Callbacks, Promises, Async/Await

Synchronous vs Asynchronous

- **Synchronous**: code top-se-bottom, **ek line khatam hue baghair agli nahi chalti** — agar koi task time leta hai, to poora program us par ruk jata hai (jaise ek hi lane mein sab logon ka ek-ek karke race complete karna).
- **Asynchronous**: time-consuming tasks (jaise data fetch karna, timers) ko **side mein bhej diya jata hai**, aur baaki program chalta rehta hai; jab wo task complete ho jaye, uska result wapas mila diya jata hai (jaise multiple lanes mein log apni-apni speed se race complete karte hain, koi kisi ka wait nahi karta).
- `setTimeout(callback, ms)` ek built-in asynchronous function hai jo `callback` ko `ms` milliseconds baad chalati hai, bina baaki code ko rokay.

Callbacks

- Callback = **ek function ko doosre function ke andar call karna**, taake unke beech ek **connection/order** ban sake (kaam A khatam ho, phir kaam B shuru ho).

- Problem: jab bohot saare async steps ek doosre ke andar nested ho jate hain (step ke andar step ke andar step...), to code ek "Christmas tree" jaisa **deeply nested, mushkil-se-parhne-wala** structure ban jata hai — ise **"callback hell"** kehte hain.

Promises

- Promise ek object hai jo kisi **future value** ko represent karta hai — abhi value maloom nahi, lekin **resolve ho jayegi (success)** ya **reject ho jayegi (failure)**.
- **Promise cycle / states:** `pending` (initial, kuch nahi hua) → `resolved`/`fulfilled` (kaam successful) ya `rejected` (kaam fail). In dono ke baad `finally` hamesha chalta hai (chahe resolve ho ya reject).

```js
function order() {
  return new Promise((resolve, reject) => {
    if (isShopOpen) {
      resolve('order details');
    } else {
      reject('shop is closed');
    }
  });
}
```

- `resolve` aur `reject` khud functions hain jo Promise executor ko diye jate hain — inhe call karke hum decide karte hain ke promise ka result kya hoga.
- **Promise chaining:** `.then()` se ek ke baad ek steps chalti hain — har `.then()` ke andar agar hum kuch **return** karte hain (jaise doosri promise), to agli `.then()` ko wahi milta hai. Isse callback hell ki jagah ek **flat, readable chain** ban jati hai:

```js
```

```
order()
  .then(() => order(2000, () => console.log('step 1')))
  .then(() => order(2000, () => console.log('step 2')))
  .catch((err) => console.log('error:', err))
  .finally(() => console.log('always runs'));
```

- `.catch()` — reject hone par error handle karta hai.
- `.finally()` — chahe resolve ho ya reject, hamesha chalta hai (jaise "shop band karna", chahe customers serve hue ho ya na hue hon).

Async/Await

- `async`/`await` promises likhne ka ek **cleaner, synchronous-jaisa** tareeqa hai.
- Function ke aage `async` keyword lagane se wo function automatically **ek Promise return karta hai.**

js

```
async function order() {
  try {
    const result = await someAsyncFunction();
    console.log(result);
  } catch (error) {
    console.log('error:', error);
  } finally {
    console.log('always runs');
  }
}
```

- `await` keyword kisi promise ke **resolve hone ka wait karta hai,** aur uski resolved value seedha ek variable mein de deta hai — `.then()` callback likhne ki zaroorat nahi rehti.

- `await` sirf `async` **function ke andar** use ho sakta hai.
- Error handling ke liye `.then()/.catch()` ki jagah `try/catch/finally` block use hota hai.
- `await` **ka real-world analogy**: chef (main thread) kitchen mein kaam kar raha hai, beech mein customer se topping poochne bahar jata hai (`await`) — us waqt kitchen ka kaam ruk jata hai, lekin **baaki restaurant ke kaam (dishes, tables, other orders) chalte rehte hain** kyunke wo alag se ho rahe hain, chef ke wapis aane ka wait nahi kar rahe.
- `then`/`catch`/`finally` handlers ko async/await ke sath bhi use kiya ja sakta hai (jaise `asyncFunc().then(...)`), lekin generally ek hi style consistently use karna behtar hai.

38. ES Modules — `export` / `import`

- Modules code ko **alag files mein organize** karne aur unke beech share karne ka tareeqa hain.

Export Karna

- **Declaration ke sath export**: `export let x = ...;`, `export function foo() {}`, `export class Bar {}` — function/class declaration ke baad semicolon zaroori nahi.
- **Declaration se alag export**: pehle normal declare karo, phir `export { name1, name2 };` se ek list export karo.
- **Default export**: har file mein **sirf ek** `export default` ho sakta hai — is se export ki gayi entity ka naam na bhi ho sakta hai (anonymous class/function).

Import Karna

- **Named imports**: `import { sayHi, sayBye } from './say.js';` — curly braces zaroori hain, aur naam **exact match** hone chahiye.
- **Default import**: `import User from './user.js';` — koi curly braces nahi, aur import karte waqt **koi bhi naam** de sakte hain (jaise `import MyUser from './user.js';` bhi chalega).

- **Import everything as object**: `import * as say from './say.js';` — phir `say.sayHi()` jaise access karna parta hai.
- **Renaming**: `import { sayHi as hi } from './say.js';` ya `export { sayHi as hi };`

Named vs Default — Tradeoffs

- Named exports **explicit** hote hain — import karte waqt exact naam use karna parta hai, jisse team ke sab members consistent naam use karte hain.
- Default exports mein import karne wala **koi bhi naam** rakh sakta hai — flexible hai lekin team ke andar inconsistency ka risk create karta hai. Isliye kuch teams sirf named exports use karna prefer karti hain.

Re-export

- `export { sayHi } from './say.js';` — kisi doosre module se import karke **turant re-export** kar dena, ek single "entry point" file banane ke liye (jaise `index.js` jo ek pura package/folder ka public interface ho).
- Default export ko re-export karne ke liye alag syntax chahiye: `export { default as User } from './user.js';`
- `export * from './module.js'` sirf **named exports** re-export karta hai, default ko nahi — dono chahiye to alag se `export { default } from './module.js';` bhi likhna parta hai.

Rules

- `import`/`export` statements **top level** par hone chahiye — kisi block (`if`, function) ke andar likhna error deta hai. Conditional/dynamic import ke liye alag syntax (dynamic imports) chahiye.

39. JSON (JavaScript Object Notation)

- JSON ek **data representation format** hai (XML/YAML jaisa), lightweight aur readable — APIs, config files, aur data storage mein widely use hota hai.

- JSON JavaScript ka **superset** hai — jo bhi JSON mein likha hai wo valid JavaScript bhi hai, isliye JS ke sath integration bohot smooth hai.
- **Supported types**: strings, numbers (decimal/negative/scientific notation sab included), booleans (`true`/`false`), `null`, arrays, aur objects (jinki values in sab types mein se koi bhi ho sakti hain).

Syntax Rules

- Object: `{ "key": value, ... }` — **har key double quotes mein honi chahiye** (single quotes allowed nahi, na hi bina quotes).
- Array: `[value1, value2, ...]`
- Sabse aakhri key-value pair ke baad **comma nahi** hona chahiye.
- Objects aur arrays ko **deeply nest** kiya ja sakta hai (object ke andar array, array ke andar object, waghera) — is se hierarchical data represent hoti hai.
- `.json` extension wali file mein sirf JSON content hota hai.

JSON aur JavaScript Ke Beech Convert Karna

- `JSON.parse(jsonString)` — JSON **string** ko JavaScript object/array mein convert karta hai (real-world mein API se data usually string ki tarah aata hai, jise parse karna parta hai).
- `JSON.stringify(jsObject)` — JavaScript object/array ko JSON **string** mein convert karta hai (server ko bhejne ke liye) — pehle bhi objects section mein cover ho chuka.

40. DOM — Traversal & Events (Deep Dive)

DOM Tree Ka Structure

- Poora DOM ek **tree of nodes** hai — elements, attributes, text content, comments, aur line breaks bhi sab **nodes** kehlate hain.
- `document` node root hai. Root ka child `<html>` hai, jiske children `<head>` aur `<body>` hain (jo aapas mein **siblings** hain), waghera.

- **HTML attributes (id, class) bhi nodes hain**, lekin ye parent-child-sibling relationship mein participate nahi karte — ye us element node ki **property** ki tarah access hote hain jispar wo lage hote hain.

Selection Methods (Recap + Comparison)

- `document.getElementById('id')` — single element, sirf ID se.
- `document.getElementsByClassName('class')` — **HTMLCollection** (array-like, direct array methods nahi chalti).
- `document.getElementsByTagName('tag')` — **HTMLCollection**, tag name se.
- `document.querySelector('css-selector')` — sirf **pehla matching element**, koi bhi CSS selector accept karta hai (id/class/tag/attribute sab).
- `document.querySelectorAll('css-selector')` — **NodeList** (array-like, `.forEach()` chal sakta hai), saare matching elements.
- **Recommendation**: `querySelector`/`querySelectorAll` zyada versatile aur modern hain — inhe prefer kiya jata hai.

DOM Manipulation (Recap + Additional Methods)

- `.style.propertyName` — inline CSS style set karna. CSS property names JS mein **camelCase** mein likhi jati hain (`font-size` → `fontSize`).
- `document.createElement('tag')` — naya element create karna (abhi tak DOM mein nahi hai).
- `.appendChild(node)` — kisi element ko **doosre element ke andar insert** karna.
- `.textContent` — sirf visible text (raw, jaisa hai waisa).
- `.innerText` — visible text, lekin CSS-aware (jaise `display:none` wala text exclude ho sakta hai).
- `.innerHTML` — HTML tags ke sath text — **security risk** hai agar user input ko directly `.innerHTML` mein daala jaye (XSS attack ka khatra), isliye zyadatar `.innerText`/`.textContent` prefer kiya jata hai.
- `.setAttribute(name, value)` / `.removeAttribute(name)` / `.getAttribute(name)` — attributes manage karna.

- `.classList.add()` / `.remove()` / `.contains()` / `.toggle()` — classes manage karna (`.toggle()` pehle bhi cover ho chuka).
- `.remove()` — element ko DOM se completely hata dena.

DOM Traversal (Parent, Child, Sibling)

- **Parent traversal**: `.parentNode` / `.parentElement` — ek level upar jana. Multiple baar chain kar ke (`.parentNode.parentNode`) aur upar ja sakte hain.
  - Farq: `.parentElement` sirf **element type** parent deta hai, agar parent element na ho (jaise `document`) to `null` deta hai. `.parentNode` **kisi bhi type** ka node de sakta hai (isliye zyada reliable hai general traversal ke liye).
- **Child traversal**: `.childNodes` (saare nodes, jisme text nodes bhi shamil — jaise HTML mein indentation ke whitespace bhi text node ban jate hain) vs `.children` (sirf **element nodes**, HTMLCollection).
  - `.firstChild`/`.lastChild` (koi bhi node type, aksar text node mil jata hai indentation ki wajah se) vs `.firstElementChild`/`.lastElementChild` (sirf element).
- **Sibling traversal**: `.previousSibling`/`.nextSibling` (koi bhi node) vs `.previousElementSibling`/`.nextElementSibling` (sirf element).
- **Practical tip**: element-specific properties (`.parentElement`, `.children`, `.firstElementChild`, `.nextElementSibling`) zyada predictable hote hain jab element par style ya kaam karna ho, kyunke text nodes (whitespace) ka masla avoid ho jata hai.

Event Listeners

- **HTML attribute method**: `<button onclick="...">` — direct HTML mein likhna. Simple hai lekin ek element par sirf **ek hi event handler** set ho sakta hai is tareeqe se (naya likhne se purana overwrite ho jata hai).
- **`addEventListener(event, callback, useCapture)`** — modern, preferred tareeqa. Ek element par **multiple listeners** add kiye ja sakte hain, aur ye event capturing/bubbling ko control karne ki flexibility bhi deta hai.

- Common events: `click`, `mouseover`, `mouseout`, `load`, `keydown`, `submit`, waghera.

Event Propagation — Capturing, Target, Bubbling

- Jab kisi element par event fire hota hai, to teen phases hoti hain:
  1. **Capturing phase**: root (window/document) se **neeche** target tak travel karta hai.
  2. **Target phase**: jis element par event actually hua, wahan trigger hota hai.
  3. **Bubbling phase**: target se wapas **upar** root tak travel karta hai (default behavior).
- `addEventListener` ka teesra parameter (`true`/`false`) decide karta hai ke listener **capturing phase mein react kare (true) ya bubbling phase mein (false, default)**.
- `event.target` — jis exact element par event fire hua, uska reference deta hai (chahe event listener kisi parent par lagi ho).
- `event.stopPropagation()` — event ko aage travel (capture ya bubble) hone se **rok deta hai**.
- `event.preventDefault()` — element ka **default browser behavior** rokta hai (jaise link ka navigate karna, ya form ka submit hoke reload hona) — pehle bhi cover ho chuka.
- `{ once: true }` option se ek listener ko sirf **ek baar** fire hone ke liye set kiya ja sakta hai.

Event Delegation

- **Pattern**: individual children par alag-alag event listeners lagane ke bajaye, ek hi listener **parent element** par laga dete hain, aur bubbling ki wajah se parent ko pata chal jata hai ke uske andar kaunse child par click hua (`event.target` se).
- **Faide**:
  1. **Less code, better performance/memory** — sirf ek listener, chahe andar 100 children hon.
  2. **Dynamically add kiye gaye future elements bhi automatically kaam karte hain** — kyunke listener parent par hai, naye add kiye gaye child elements ke

liye alag se listener lagane ki zaroorat nahi (jab tak wo parent ke andar hi hon).

- Implementation mein `event.target.matches('selector')` se check karte hain ke click sahi type ke child par hua ya nahi, phir us par action lete hain.

41. Modern JavaScript — Quick Reference (Pro Tips)

Debugging Tools (Console)

- `console.log({ variableName })` — **computed property shorthand** use karke object mein daalna, taake console mein variable ka **naam bhi dikhe**, sirf value nahi.
- `console.log('%c text', 'css-string')` — console output ko custom **CSS se style** karna.
- `console.table(arrayOfObjects)` — array of objects ko ek **readable table** format mein dikhata hai.
- `console.time('label')` / `console.timeEnd('label')` — code ka **execution time measure** karna.
- `console.trace()` — batata hai ke function **kahan se define hua aur kahan-kahan se call hua** (stack trace) — debugging ke liye useful jab pata karna ho ke koi function accidentally do baar to nahi call ho raha.

Object Destructuring (Recap + Function Parameters)

- Function parameters mein bhi object destructure kar sakte hain — poori object variable pass karne ke bajaye, sirf zaroori properties directly parameter list mein le lete hain:

```js
function feed({ name, food }) {
  return `${name} eats ${food}`;
}
```

- Ye code ko clean karta hai — object ka naam baar-baar (`obj.prop1`, `obj.prop2`) likhne ki zaroorat nahi rehti.

Tagged Template Literals (Advanced)

- Template literal ko ek function ke sath directly "attach" kar sakte hain (bina parentheses ke) — function ko string ke **static parts** aur **interpolated values** dono alag-alag milte hain, jinse custom string-building logic likhi ja sakti hai. Ye advanced templating libraries mein use hota hai.

Spread Syntax

- **Objects merge karna:** `const merged = { ...obj1, ...obj2 };` — right wali properties left ko override karti hain agar same key ho. Ye `Object.assign()` ka modern, cleaner alternative hai.
- **Arrays extend karna:** `const newArr = [...arr, newItem1, newItem2];` — end mein add karne ke liye `.push()` ka alternative. `[newItem, ...arr]` se start mein bhi add ho sakta hai.
- **Trailing comma**: array/object ki aakhri item ke baad comma lagana ab **valid aur good practice** consider hoti hai (version control diffs cleaner rehte hain).

`Array.prototype.reduce()`

- `.reduce()` ek array ko **single value** mein accumulate karta hai — callback ko `(accumulator, currentValue)` milte hain:

```
js

const total = orders.reduce((sum, order) => sum + order, 0);
```

- Ye loops se zyada declarative aur concise hota hai jab kisi array se ek final combined value nikalni ho (sum, average, ya koi bhi custom accumulation).

Active Recall — Q&A

1. **Q:** JavaScript "interpreted" language kyun kehlati hai? **A:** Kyunke ye compiler ke bagair directly execute hoti hai, Java jaisi compiled language nahi hai.

2. **Q:** `var`, `let`, aur `const` mein kya farq hai? **A:** `var` globally scoped hai aur conflicts create kar sakta hai (ab avoid karte hain). `let` reassignable hai. `const` reassign nahi ho sakta aur declare karte waqt value dena zaroori hai.
3. **Q:** `typeof null` kya return karta hai aur kyun ye confusing hai? **A:** `"object"` return karta hai — ye JS ka ek legacy bug/quirk hai, technically galat hai.
4. **Q:** Template literals kaise likhte hain aur inka fayda kya hai? **A:** Backticks (`` ` ``) use karke `${variable}` syntax se variables directly string mein embed kar sakte hain — old concatenation (`+`) se zyada readable hai.
5. **Q:** `==` aur `===` mein kya farq hai? **A:** `==` sirf value compare karta hai, `===` value aur type dono compare karta hai.
6. **Q:** Array mein zero-based indexing ka matlab kya hai? **A:** Array ka pehla element index `0` par hota hai, doosra `1` par, waghera.
7. **Q:** `.map()`, `.filter()`, aur `.forEach()` mein kya farq hai? **A:** `.forEach()` sirf loop chalata hai, kuch return nahi karta. `.map()` transform kiya hua naya array return karta hai. `.filter()` condition match karne wale items ka naya array return karta hai.
8. **Q:** Object destructuring kya hoti hai? **A:** Object ki properties ko seedha variables mein nikaalna, e.g. `const { firstName, lastName } = person;`
9. **Q:** JSON aur JS object literal mein main farq kya hai? **A:** JSON mein saari keys aur string values **double quotes** mein honi chahiye; single quotes allowed nahi.
10. **Q:** Arrow function ka short syntax kab use kar sakte hain? **A:** Jab function mein sirf ek expression ho, to curly braces aur `return` drop kar sakte hain; agar sirf ek parameter ho to parenthesis bhi drop ho sakti hai.
11. **Q:** Constructor function mein `prototype` par method add karne ka fayda kya hai? **A:** Method har object instance mein duplicate nahi hoti, memory-efficient hoti hai — sab instances usi ek prototype method ko share karte hain.
12. **Q:** ES6 classes ko "syntactic sugar" kyun kaha jata hai? **A:** Kyunke peeche se wo bhi prototypes hi use karti hain — bas likhne ka tareeqa cleaner aur readable hai, functionality same hai.

13. **Q:** `querySelector` aur `querySelectorAll` mein kya farq hai? **A:** `querySelector` sirf pehla matching element return karta hai. `querySelectorAll` saare matching elements ka NodeList return karta hai (jispar array methods chal sakti hain).
14. **Q:** Form submit event mein `preventDefault()` kyun call karte hain? **A:** Taake form ka default behavior (page reload/navigate) na ho, aur hum apna custom JS logic handle kar sakein.
15. **Q:** `NodeList` aur `HTMLCollection` mein kya farq hai? **A:** `NodeList` (querySelectorAll se) par array methods (jaise forEach) direct chal sakti hain. `HTMLCollection` (getElementsByClassName/TagName se) par array methods chalane ke liye manually array mein convert karna parta hai.
16. **Q:** Object literal notation se object kaise banate hain? **A:** Curly braces `{}` use karke, andar key-value pairs comma se separate karke likhte hain, e.g. `{ name: 'Crystal', age: 30 }`.
17. **Q:** Dot notation aur square bracket notation mein kya farq hai object properties access karne mein? **A:** Dot notation (`user.name`) fixed/static property names ke liye use hoti hai. Square bracket notation (`user['name']` ya `user[key]`) dynamic keys ke liye use hoti hai — jab property ka naam kisi variable mein store ho.
18. **Q:** Object mein method kaise define karte hain? **A:** Key ki value ek function rakh ke, e.g. `login: function() { ... }` ya shorthand syntax `login() { ... }`.
19. **Q:** `this` keyword ka matlab kya hai aur ye kis par depend karta hai? **A:** `this` current execution context ko refer karta hai. Global scope mein `this` = `window` object. Kisi object ke method (regular function) ke andar `this` = wo object jispar method call hua.
20. **Q:** Arrow function object ke method ke andar `this` ke sath kyun kaam nahi karti? **A:** Kyunke arrow function `this` ki value ko object se bind nahi karti — wo surrounding/current context ki `this` value use karti hai (jo usually `window` hoti hai), isliye object ki property access nahi kar pati.
21. **Q:** Array of objects pattern kya hai aur ye kyun common hai? **A:** Ye ek array hai jiska har element ek object hota hai (jaise `[{title: '...', likes: 30}, ...]`). Real-world data (jaise database se aane wala data) mostly isi format mein hota hai.

22. **Q:** `Math.floor()`, `Math.ceil()`, aur `Math.round()` mein kya farq hai? **A:** `Math.floor()` hamesha neeche wale integer tak round karta hai, `Math.ceil()` hamesha upar wale tak, aur `Math.round()` nearest integer tak (jo bhi kareeb ho).
23. **Q:** `Math.random()` se 1 aur 100 ke beech random number kaise nikalte hain? **A:** `Math.round(Math.random() * 100)` — kyunke `Math.random()` khud 0 aur 1 ke beech decimal deta hai.
24. **Q:** Primitive types aur reference types mein memory storage ka kya farq hai? **A:** Primitive values (number, string, boolean, null, undefined, symbol) **stack** memory mein store hoti hain. Reference types (objects, arrays, functions) **heap** memory mein store hote hain, aur stack par sirf unka **pointer** hota hai.
25. **Q:** Agar ek object ko doosre variable mein copy karein (`const user2 = user1`), aur phir `user2` ki property change karein, to kya `user1` bhi change hoga? Kyun? **A:** Haan, change hoga — kyunke object copy karne par sirf **pointer** copy hota hai, dono variables **same object** ko heap par point karte hain, alag copy nahi banti.
26. **Q:** OOP ke four core pillars kya hain? **A:** Encapsulation (related data/functions ko object mein group karna), Abstraction (complex details hide karna, sirf essentials expose karna), Inheritance (redundant code khatam karna, common properties/methods share karna), Polymorphism (ek method ka behavior object ke type ke hisaab se alag hona, switch/case avoid karna).
27. **Q:** Factory function aur constructor function mein basic farq kya hai? **A:** Factory function ek regular function hai jo object **return** karta hai. Constructor function `this` keyword aur `new` operator use karta hai object banane ke liye — naming convention mein pehla letter capital hota hai.
28. **Q:** `new` operator use karne par teen cheezein kya hoti hain? **A:** (1) Ek naya empty object create hota hai, (2) `this` us naye object ko point karta hai, (3) wo object function se automatically return ho jata hai.
29. **Q:** Agar constructor function ko `new` ke bina call karein to kya hoga? **A:** `this` global object (`window` browser mein) ko point karega, aur properties galti se global object par ban jayengi — jo ek bad practice/bug ka source hai.

30. **Q:** Function ko primitive value pass karne aur object pass karne mein kya farq hai? **A:** Primitive pass karne par uski **value copy** hoti hai (function ke andar change karne se bahar wali value affect nahi hoti). Object pass karne par uska **reference copy** hota hai (function ke andar object ki property change karne se bahar wala original object bhi change ho jata hai).
31. **Q:** Object mein private members kaise banate hain? **A:** Un values ko object ki property (`this.x`) banane ke bajaye, constructor function ke andar sirf **local variable** ke tor par define karte hain — is se wo bahar se access nahi ho paate, lekin closure ki wajah se object ke andar defined methods unhe access kar sakte hain.
32. **Q:** Closure kya hoti hai aur scope se kaise alag hai? **A:** Scope temporary hoti hai (function khatam hone par local variables die ho jate hain). Closure ek inner function ko apne parent function ke variables **yaad rakhne** deti hai, chahe parent function execute ho chuka ho — is se wo variables memory mein preserve rehte hain.
33. **Q:** `Object.defineProperty()` ka `get` aur `set` kya karte hain? **A:** `get` function property ko **read** karne ke liye use hota hai (property ki tarah access hoti hai, method ki tarah call nahi karni parti). `set` function property ko **update** karne ke liye use hota hai, aur ismein validation logic bhi likh sakte hain.
34. **Q:** `var` aur `let`/`const` mein scope ka kya farq hai? **A:** `var` sirf **function-scoped** hota hai (block ke curly braces ko ignore kar deta hai). `let` aur `const` **block-scoped** hote hain — sirf usi block tak mehdood rehte hain jismein declare hue.
35. **Q:** `var` ko dobara declare karne aur `let`/`const` ko dobara declare karne mein kya farq hai? **A:** `var` same naam se dobara declare karne par koi error nahi aata (bas overwrite ho jata hai). `let`/`const` dobara declare karne par error aata hai ("already been declared") — ye desirable hai kyunke bugs jaldi pakre jaate hain.
36. **Q:** `while` loop aur `do-while` loop mein kya farq hai? **A:** `while` loop condition **pehle check** karta hai, phir block chalata hai (agar condition false ho to block ek baar bhi nahi chalega). `do-while` block **pehle chalata hai**, phir condition check karta hai — isliye block kam se kam **ek baar zaroor chalta hai**, chahe condition shuru se false ho.

37. **Q:** `for...in` aur `for...of` mein kya farq hai? **A:** `for...in` object/array ki **keys ya indexes** deta hai. `for...of` array (ya iterable) ki seedhi **values** deta hai.
38. **Q:** Infinite loop kyun banta hai aur iska kya nuksan hai? **A:** Agar loop ki condition kabhi false na ho (jaise `while` loop mein increment/decrement bhool jayein), to loop hamesha chalta rehta hai — ye application ya browser ko crash/freeze kar sakta hai.
39. **Q:** camelCase naming convention kya hai? **A:** Multi-word variable naam likhte waqt pehla word chhota (lowercase) rakhte hain aur baaki words ke pehle letter ko capital karte hain, e.g. `userName`.
40. **Q:** Primitive values "immutable" kyun kehlati hain, aur variable reassignment se ye kaise alag hai? **A:** Primitive value khud kabhi change/mutate nahi hoti — jab hum variable ko naya value dete hain, to variable sirf ek naye (alag) value ko point karne lagta hai, purani value untouched rehti hai. Ye objects/arrays se alag hai, jinki properties directly mutate ki ja sakti hain bina naya object banaye.
41. **Q:** `break` aur `continue` mein kya farq hai? **A:** `break` poore loop ko turant terminate kar deta hai. `continue` sirf current iteration skip karta hai, loop agli iteration se chalta rehta hai.
42. **Q:** Labeled statement kis liye use hoti hai? **A:** Nested loops mein `break`/`continue` ko specify karne ke liye ke wo kaunse (outer ya inner) loop ko target kar raha hai — label naam de ke `break labelName;` likh sakte hain.
43. **Q:** JavaScript mein kaunsi values "falsy" treat hoti hain? **A:** `false`, `undefined`, `null`, `0`, `NaN`, aur empty string (`''`). In ke ilawa baaki sab kuch truthy hota hai.
44. **Q:** `if (x === 5 || 7 || 10)` likhna kyun galat hai? **A:** Kyunke `7` aur `10` khud truthy values hain, isliye ye condition hamesha `true` return karegi — sahi tareeqa har condition poori likhna hai: `if (x === 5 || x === 7 || x === 10)`.
45. **Q:** `switch` statement mein `break` bhoolne se kya hota hai? **A:** Matching case ke baad wale saare cases bhi chalte rahenge (fall-through), jab tak koi `break` na mile ya switch khatam na ho jaye.
46. **Q:** Array technically kis data type ka hissa hai? **A:** Array ek **object** hai (special type ka) — JS ke basic data types mein array alag se shamil nahi, ye object hi hai jispar

special array methods aur `length` property available hoti hai.

47. **Q:** `push`/`pop` `shift`/`unshift` se fast kyun hote hain? **A:** `push`/`pop` sirf array ke **end** par kaam karte hain, baaki elements ko renumber nahi karna parta. `shift`/`unshift` array ke **start** par kaam karte hain, isliye baaki saare elements ko re-index (renumber) karna parta hai — jo slow hai.
48. **Q:** `array.length = 0` se kya hota hai? **A:** Array ko **clear** kar deta hai (truncate) — ye array clear karne ka simplest tareeqa hai.
49. **Q:** `.slice()` array ke objects ke sath copy karte waqt kya dhyan rakhna chahiye? **A:** `.slice()` sirf **references copy karta hai**, actual objects nahi — isliye original aur copied array ke objects **same memory** ko point karte hain; ek array ke object ki property change karne se doosre array mein bhi wo change nazar aayega.
50. **Q:** `.indexOf()` objects ke liye kaam kyun nahi karta? **A:** Kyunke objects **reference type** hain — do objects jo dikhne mein same hon, unke memory references alag ho sakte hain, isliye exact match nahi milta. Objects ke liye `.findIndex()` ya `.find()` use karte hain (callback-based).
51. **Q:** `.map()` aur `.filter()` mein kya farq hai (recap + emphasis)? **A:** `.map()` har element ko **transform** karke naya array deta hai (same length). `.filter()` sirf condition pass karne wale elements ka naya array deta hai (length kam ho sakti hai). Dono original array ko change nahi karte.
52. **Q:** `.some()` aur `.every()` mein kya farq hai? **A:** `.some()` true deta hai agar array ka **kam se kam ek** item condition pass kare. `.every()` true deta hai sirf tab jab **saare** items condition pass karein.
53. **Q:** Arrays ko `==` se compare karna kyun galat hai? **A:** Kyunke arrays reference types hain — `==` sirf tab true dega jab dono variables **same array object** ko point karein, chahe unke andar values same hi kyun na hon (e.g. `[] == []` hamesha `false` hai).
54. **Q:** `.sort()` method destructive kyun kehlata hai? **A:** Kyunke ye naya array return nahi karta, balke **original array ko directly modify** kar deta hai.

55. **Q:** `for...in` loop arrays ke liye recommend kyun nahi kiya jata? **A:** Kyunke ye sirf numeric indexes tak mehdood nahi rehta — agar array mein koi custom (non-numeric) property add ki gayi ho, to `for...in` wo bhi include kar deta hai, jo unexpected results de sakta hai. Arrays ke liye traditional `for` ya `for...of` behtar hain.
56. **Q:** Parameter aur argument mein kya farq hai? **A:** Parameter function declare karte waqt use hone wala variable naam hai. Argument function call karte waqt di gayi actual value hai.
57. **Q:** `return` statement ke sath ASI (Automatic Semicolon Insertion) ka gotcha kya hai? **A:** Agar `return` ke baad value ko **naye line** par likhein, to JS automatically `return` ke baad semicolon insert kar deta hai, jisse function `undefined` return karega aur intended value kabhi return nahi hogi. `return` aur value hamesha same line par honi chahiye.
58. **Q:** Function declaration aur function expression mein kya farq hai? **A:** Function declaration naam ke sath directly define hoti hai (`function greet() {}`). Function expression ek anonymous function ko variable mein store karti hai (`const greet = function() {}`).
59. **Q:** Callback function kya hoti hai? **A:** Ek function jo doosre function ko **parameter ki tarah pass** ki jati hai — receiving function apni marzi se decide karta hai ke callback ko kab aur kaise call karna hai.
60. **Q:** IIFE kya hai aur ye kis liye use hota hai? **A:** Immediately Invoked Function Expression — ek function jo create hote hi turant khud call ho jata hai (parentheses mein wrap karke). Libraries mein ye pattern use hota hai taake internal variables function scope ki wajah se bahar se access na ho sakein (privacy).
61. **Q:** Arrow function mein parentheses aur curly braces kab omit (hata) sakte hain? **A:** Parentheses sirf tab jab function ka sirf ek simple parameter ho. Curly braces (aur `return`) sirf tab jab function body mein sirf ek expression directly return ho raha ho.
62. **Q:** Arrow function se object literal return karte waqt kya dhyan rakhna chahiye? **A:** `() => { foo: 1 }` kaam nahi karega, kyunke JS `{` ko function body (block)

samajh leta hai. Object return karne ke liye parentheses mein wrap karo: `() => ({ foo: 1 })`.

63. **Q:** Arrow functions object methods ke tor par use nahi karni chahiye — kyun? **A:** Kyunke arrow function ki apni `this` binding nahi hoti — ye surrounding/lexical scope se `this` leti hai, isliye object ke andar method banane par `this` object ko refer nahi karega (window/global object ko refer karega).
64. **Q:** `call()`, `apply()`, aur `bind()` teeno ka common maqsad kya hai, aur inmein farq kya hai? **A:** Teeno function ke andar `this` ki value manually set karte hain. `call()` turant call karta hai aur arguments individually deta hai. `apply()` turant call karta hai lekin arguments array mein deta hai. `bind()` turant call nahi karta — naya function return karta hai jise baad mein manually call karna parta hai.
65. **Q:** "Function borrowing" kya hoti hai? **A:** Kisi ek object ke method ko `call()`/`apply()` ke through kisi **doosre object** ke context mein use karna, chahe us doosre object mein wo method khud na ho.
66. **Q:** "Function currying" `bind()` se kaise implement hoti hai? **A:** `bind()` ko use karke function ke kuch parameters **pehle se fix** kar dete hain (e.g. `multiply.bind(null, 2)`), taake naya function banaye jaisa jismein sirf baaki parameters dene parein.
67. **Q:** Event listener ke andar class method use karne par `this` ka kya masla aata hai? **A:** Method ke andar `this` class ke object ko refer karne ke bajaye us **HTML element** ko refer karta hai jispar event fire hua — kyunke event listener callback ko us element ke context mein invoke karta hai.
68. **Q:** Event listener wale `this` masle ko kaise fix karte hain? **A:** Constructor ke andar method ko explicitly bind karke (`this.method = this.method.bind(this)`), ya event listener ke andar method ko ek arrow function se wrap karke call karke.
69. **Q:** Arrow functions class fields mein "auto-bound methods" kyun kehlate hain? **A:** Kyunke class field mein arrow function likhne se wo automatically instance ke `this` ko lexically capture kar leti hai — isse manually `bind()` karne ki zaroorat nahi parti, jo regular methods ke sath karna parta hai.

70. **Q:** `arguments` object arrow functions mein kyun nahi hota? **A:** Arrow functions ki apni `arguments` binding nahi hoti — agar arrow function ke andar `arguments` use karein, to wo uske **surrounding/enclosing function** ke arguments ko refer karega. Isliye arrow function mein arguments ke liye rest parameters (`...args`) use karna behtar hai.
71. **Q:** `this` ki value kis cheez se decide hoti hai — definition se ya call se? **A:** Call se — `this` ki value function **kaise call hua** usi se decide hoti hai (runtime binding), function kahan define hua usse nahi.
72. **Q:** Promise ke teen states kya hain? **A:** `pending` (initial, kuch nahi hua), `resolved`/`fulfilled` (successful), aur `rejected` (fail hua).
73. **Q:** `async`/`await` ka fayda callbacks/promises ke muqable mein kya hai? **A:** Ye asynchronous code ko **synchronous jaisi readable shakal** mein likhne deta hai — `.then()` chains ki jagah seedha `await` se value milti hai, aur error handling `try/catch` se hoti hai, jo deeply-nested callback hell se bohot cleaner hai.
74. **Q:** `await` sirf kahan use ho sakta hai? **A:** Sirf `async` function ke andar.
75. **Q:** Named export aur default export mein import karte waqt kya farq hai? **A:** Named export import karte waqt **exact naam** curly braces ke sath use karna parta hai (`import { User }`). Default export import karte waqt **koi bhi naam** de sakte hain, curly braces ke bagair (`import Anything from ...`).
76. **Q:** Ek file mein kitne default exports ho sakte hain? **A:** Sirf ek.
77. **Q:** JSON mein object keys kis tarah likhni zaroori hain? **A:** Hamesha **double quotes** mein — single quotes ya bina quotes ke JSON invalid ban jata hai.
78. **Q:** `JSON.parse()` aur `JSON.stringify()` mein kya farq hai? **A:** `JSON.parse()` JSON string ko JS object mein convert karta hai. `JSON.stringify()` JS object ko JSON string mein convert karta hai.
79. **Q:** `.parentNode` aur `.parentElement` mein kya farq hai? **A:** `.parentElement` sirf element-type parent deta hai (element na ho to `null`). `.parentNode` kisi bhi type ka node de sakta hai, isliye zyada reliable hai.

80. **Q:** `.childNodes` aur `.children` mein kya farq hai? **A:** `.childNodes` saare nodes deta hai (text nodes/whitespace bhi shamil). `.children` sirf element nodes deta hai.
81. **Q:** Event propagation ki teen phases kya hain? **A:** Capturing (root se target tak neeche), Target (jahan event actually hua), aur Bubbling (target se wapas root tak upar) — bubbling default hoti hai.
82. **Q:** `addEventListener` ka teesra parameter (`true`/`false`) kya control karta hai? **A:** Ye decide karta hai ke listener capturing phase mein react kare (`true`) ya bubbling phase mein (`false`, default).
83. **Q:** `event.stopPropagation()` aur `event.preventDefault()` mein kya farq hai? **A:** `stopPropagation()` event ko aage travel (capture/bubble) hone se rokta hai. `preventDefault()` element ka default browser behavior rokta hai (jaise link navigate karna ya form submit hoke reload hona).
84. **Q:** Event delegation kya hai aur iske do main faide kya hain? **A:** Ek hi event listener ko parent element par lagana (har child par alag lagane ke bajaye), bubbling ki wajah se target check karke action lena. Faide: (1) kam code/better performance-memory, (2) dynamically future mein add hone wale child elements bhi automatically kaam karte hain, alag se listener lagane ki zaroorat nahi.
85. **Q:** `.innerHTML` use karte waqt kya security risk hota hai? **A:** Agar user input directly `.innerHTML` mein daala jaye, to wo malicious script inject kar sakta hai (XSS attack) — isliye plain text ke liye `.innerText`/`.textContent` zyada safe hain.
86. **Q:** Function parameters mein object destructuring ka fayda kya hai? **A:** Poori object variable pass karne ke bajaye sirf zaroori properties directly parameter list mein le sakte hain, jisse function body mein baar-baar `obj.property` likhne ki zaroorat nahi rehti — code zyada clean ban jata hai.
87. **Q:** Spread syntax se objects merge karte waqt agar dono mein same key ho to kya hota hai? **A:** Jo object baad mein (right side) spread hua ho, uski value **override** kar deti hai pehle wali (left side) ki value ko.
88. **Q:** `.reduce()` method kya karta hai? **A:** Poore array ko ek **single accumulated value** mein convert karta hai — callback ko `(accumulator, currentValue)` milte hain, jinse sum, average, ya koi bhi custom combination nikali ja sakti hai.

89. **Q:** `console.table()` kis liye useful hai? **A:** Array of objects ko ek readable, columns-wale **table format** mein console mein dikhane ke liye.
90. **Q:** Callback hell kya hota hai aur iska solution kya hai? **A:** Jab bohot saare async steps ek doosre ke andar deeply nested ho jate hain, code padhna mushkil ho jata hai ("Christmas tree" shape). Solution: Promises (chaining ke sath flat structure) ya async/await (synchronous-jaisi readable code).
91. **Q:** Trailing comma (array/object ki last item ke baad comma) modern JS mein kyun acceptable hai? **A:** Ye syntax error nahi deta, aur version control diffs ko cleaner rakhta hai (naya item add karne par sirf ek line change hoti hai, purani line mein comma add nahi karna parta).
92. **Q:** `querySelectorAll` se milne wali NodeList aur `getElementsByClassName` se milne wali HTMLCollection mein kya farq hai? **A:** NodeList par array methods (jaise `.forEach()`) direct chal sakti hain. HTMLCollection par nahi chalti, usko pehle array mein convert karna parta hai.
93. **Q:** DOM tree mein HTML attributes (id, class) nodes to hote hain, lekin ye parent-child relationship mein kyun participate nahi karte? **A:** Ye us element node ki **property** ki tarah access hote hain (jaise `element.id`), khud ek alag tree branch nahi banate jaisa child elements banate hain.
94. **Q:** `{ once: true }` option `addEventListener` mein kya karta hai? **A:** Listener ko sirf **ek baar** fire hone ke liye set kar deta hai — pehli baar chalne ke baad wo automatically remove ho jata hai.
95. **Q:** `export * from './module.js'` se kya export nahi hota? **A:** Sirf named exports re-export hoti hain — **default export re-export nahi hoti**, uske liye alag se `export { default } from './module.js';` likhna parta hai.